DP-LMI package
User Manual

Version v0.3.1

July 2026

Abstract

This manual teaches the DP-LMI package from the continuous matrix inequality to the finite semidefinite program. A worked 2-by-2 LPV Lyapunov problem first introduces physical grids, local Bernstein coefficients, matrix algebra, parameter-rate vertices, certificate selection, and the YALMIP boundary. The reference chapters then document `pdbase`, `pdmat`, `pdvar`, and `pdlmi`. Two self-contained appendices develop the Bernstein and polynomial-positivity background for readers who know basic matrices and calculus but have not previously used Bernstein bases or SOS certificates.

Contents

1

From a DP-LMI to Solver Constraints

1.1 Begin With The Continuous Inequality

Let the scheduling vector be $\boldsymbol{\rho} = (\rho_1, \ldots, \rho_\ell) \in \mathcal{P}$ and let its rate belong to a box $\dot{\boldsymbol{\rho}} \in \mathcal{V}$. The package targets the differentiable parameter-dependent LMI

$$\sum_i R_i(\boldsymbol{\rho}) \left(\sum_{j=1}^{\ell} \frac{\partial x_i(\boldsymbol{\rho})}{\partial \rho_j} \dot{\rho}_j \right) + F_0(\boldsymbol{\rho}) + \sum_i F_i(\boldsymbol{\rho}) x_i(\boldsymbol{\rho}) \prec 0. \tag{1}$$

The known functions are the R_i and F_i; the differentiable functions x_i are decisions. After known data are fixed, every term must remain affine in the decision coefficients. If the derivative term is absent, then eq. (1) is an ordinary parameter-dependent LMI (PD-LMI). Thus a PD-LMI is the derivative-free special case of the DP-LMI treated here.

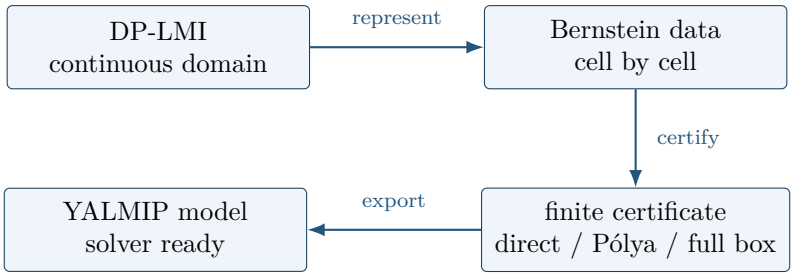
A concrete one-parameter example is the LPV Lyapunov condition

$$P(\rho) \succeq I, \tag{2}$$

$$\dot{P}(\rho, \dot{\rho}) + P(\rho)A(\rho) + A(\rho)^{\mathsf{T}} P(\rho) \preceq -\varepsilon I, \tag{3}$$

$$\dot{P}(\rho, \dot{\rho}) = \frac{\partial P(\rho)}{\partial \rho} \dot{\rho}. \tag{4}$$

Taking $\dot{\rho} = 0$, or removing $\dot{P}$, gives the familiar PD-LMI Lyapunov inequality. In the worked path below P is 2-by-2, $\rho \in [0, 1]$, $\dot{\rho} \in [-1, 1]$, and $\varepsilon = 10^{-6}$. The package inserts no hidden strictness margin; the displayed εI is part of the user's model.



The package changes representation and certificate; it does not replace YALMIP's solver interface.

1.2 Separate Physical Nodes, Cells, And Local Labels

For one parameter choose physical grid nodes

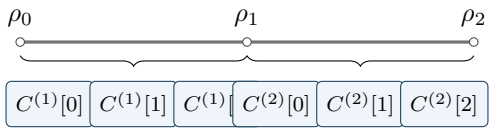
$$\rho_0 < \rho_1 < \rho_2.$$

The subscript on ρ_k identifies a physical node only. The two physical cells have separate identities $c = 1$ and $c = 2$: $[\rho_0, \rho_1]$ and $[\rho_1, \rho_2]$. On cell c, define the forward coordinate

$$\alpha = \frac{\rho - \rho_{c-1}}{\rho_c - \rho_{c-1}} \in [0, 1].$$

For local degree $m = 2$, the three coefficient labels are bracketed local positions, not additional physical nodes:

$$M_c(\alpha) = \sum_{i=0}^{2} B_i^2(\alpha) C^{(c)}[i].$$



physical-node subscripts (k), cell identity (c), and local labels ([i]) have different jobs

In MATLAB, the physical grid and degree are likewise independent choices. The example below uses degree one, so each cell has labels ([0]) and ([1]).

MATLAB input

```
M = pdmat([0 0.5 1], {10, 20, 30}, Degree=1);
disp(M.cells())
disp(M.lbls())
fprintf("cells=%d, coefficients per cell=%d\n", M.ncell(), M.ncoeff());
c1 = M.coeffs(1);
c2 = M.coeffs(2);
disp([c1{:}])
disp([c2{:}])
```

Output

```
     1
     2

     0
     1

cells=2, coefficients per cell=2
    10    20
    20    30
```

The shared value 20 belongs to the physical interface ρ_1. It is seen as $C^{(1)}[1]$ from the left cell and $C^{(2)}[0]$ from the right. This is why a physical-node index must not be reused as a local coefficient label.

1.3 Store Known Data And Decisions On The Grid

The known LPV matrix in the worked example is affine in ρ. Supplying `Degree=1` asks `pdmat` to retain exact function evaluation and verified Bernstein coefficient evidence.

MATLAB input

```
yalmip('clear')
grid = {[0 1]};
A = pdmat(grid, @(rho) [-1, 0.5; -1, -2] + ...
    rho * [-1.3, -20; 2, -10], Degree=1);
fprintf("A: %d-by-%d, cells=%d, degree=%d, coefficients/cell=%d\n", ...
    size(A,1), size(A,2), A.ncell(), A.Degree, A.ncoeff());
disp(A.evaluate(0))
```

Output

```
A: 2-by-2, cells=1, degree=1, coefficients/cell=2
   -1.0000    0.5000
   -1.0000   -2.0000
```

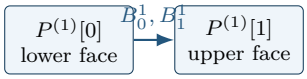
The decision $P(\rho)$ uses the same physical grid. Its two endpoint coefficient matrices are YALMIP variables; `pdvar` shares boundary handles automatically when a grid has several cells.

MATLAB input

```
P = pdvar(2, grid, "symmetric");
fprintf("P: %d-by-%d, degree=%d, continuous=%d\n", ...
    size(P,1), size(P,2), P.Degree, P.IsContinuous);
fprintf("local coefficient matrices=%d\n", numel(P.coeffs(1)));
```

Output

```
P: 2-by-2, degree=1, continuous=1
local coefficient matrices=2
```



a coefficient is a full 2-by-2 matrix, not a sampled scalar

1.4 Form Matrix Algebra In Coefficient Space

Fix a physical cell (c). Its identity remains in the parenthesized superscript while the bracketed integers are local Bernstein labels: $P = \sum_i B_i^m P^{(c)}[i]$ and $A = \sum_j B_j^n A^{(c)}[j]$. Their ordered product is

$$PA = \sum_{k=0}^{m+n} B_k^{m+n} R^{(c)}[k], \qquad R^{(c)}[k] = \sum_{i+j=k} \frac{\binom{m}{i}\binom{n}{j}}{\binom{m+n}{k}} P^{(c)}[i] A^{(c)}[j].$$

For two degree-one factors this gives

$$R^{(c)}[0] = P^{(c)}[0]A^{(c)}[0], \qquad R^{(c)}[1] = \tfrac{1}{2}\Big(P^{(c)}[0]A^{(c)}[1] + P^{(c)}[1]A^{(c)}[0]\Big),$$

$$R^{(c)}[2] = P^{(c)}[1]A^{(c)}[1].$$

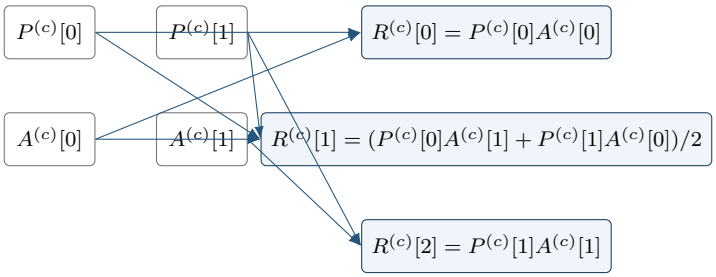
Matrix order is preserved. The product `P*A` therefore has degree two; it is not formed by multiplying a few samples.

MATLAB input

```
PA = P * A;
ATP = A' * P;
fprintf("degree(P*A)=%d, degree(A'*P)=%d\n", PA.Degree, ATP.Degree);
fprintf("coefficient matrices in P*A=%d\n", numel(PA.coeffs(1)));
```

Output

```
degree(P*A)=2, degree(A'*P)=2
coefficient matrices in P*A=3
```



1.5 Differentiate And Enumerate Rate Vertices

For a degree-m cell (c) of width h_c, differentiation is a forward difference of adjacent local coefficients:

$$\frac{\partial P}{\partial \rho} = \frac{m}{h_c} \sum_{i=0}^{m-1} (P^{(c)}[i+1] - P^{(c)}[i]) B_i^{m-1}(\alpha).$$

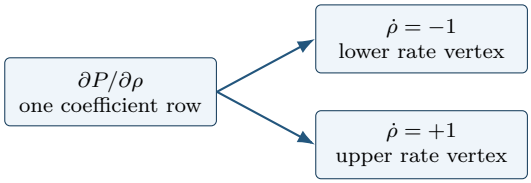
Because $\dot{P} = (\partial P/\partial \rho)\dot{\rho}$ is affine in the rate, checking both vertices -1 and $+1$ covers the whole interval $\dot{\rho} \in [-1, 1]$.

MATLAB input

```
D = rhodiff(P, [-1 1]);
fprintf("derivative degree=%d, continuous=%d\n", D.Degree, D.IsContinuous);
fprintf("rate rows=%d, coefficients per row=%d\n", ...
    size(D.coeffs(1),1), size(D.coeffs(1),2));
```

Output

```
derivative degree=0, continuous=0
rate rows=2, coefficients per row=1
```



rate rows are not physical cells and are not extra Bernstein labels

1.6 Compare The Residual And Choose A Certificate

The following MATLAB expressions are a direct transcription of eqs. (2) and (3). Addition elevates the degree-zero derivative rows to the degree-two algebraic residual without changing their represented values.

MATLAB input

```
residual = D + P * A + A' * P;
Cdecay = residual <= -1e-6 * eye(2);
Cpositive = P >= eye(2);
fprintf("residual degree=%d\n", residual.Degree);
fprintf("direct constraints: decay=%d, positivity=%d\n", ...
    numel(Cdecay.Constraints), numel(Cpositive.Constraints));
```

Output

```
residual degree=2
direct constraints: decay=6, positivity=2
```

There is one physical cell, two rate vertices, and three degree-two local labels, hence $1 \times 2 \times 3 = 6$ decay constraints. Positivity has no rate rows and uses the two degree-one coefficients of P.

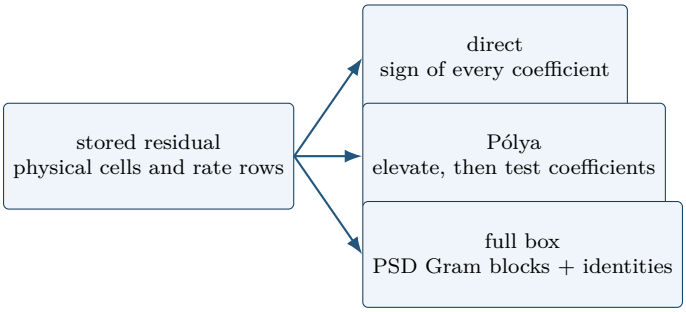
Direct coefficient constraints are the default. Two opt-in alternatives keep the same stored residual but change the finite sufficient certificate:

MATLAB input

```
Cpolya = Cdecay.applyPolya(1);
Cbox = Cdecay.applyFullBoxPreorder();
fprintf("direct=%d, Polya(1)=%d, full-box=%d\n", ...
    numel(Cdecay.Constraints), numel(Cpolya.Constraints), ...
    numel(Cbox.Constraints));
```

Output

```
direct=6, Polya(1)=8, full-box=10
```



A failed sufficient certificate does not prove that the continuous inequality is infeasible. Appendix B derives the three implemented choices and places them beside SOS background.

1.7 Export The Finite Model To YALMIP

`toYalmip` returns exactly the stored finite constraints. It does not select a solver, add an objective, or insert a margin.

MATLAB input

```
Fdecay = toYalmip(Cdecay);
Fpositive = Cpositive.toYalmip();
fprintf("exported constraints: decay=%d, positivity=%d\n", ...
    length(Fdecay), length(Fpositive));
```

Output

```
exported constraints: decay=6, positivity=2
```

Use ordinary YALMIP calls after that boundary:

MATLAB input

```
solver = 'lmilab';
if exist('mosekopt', 'file') ~= 0
    solver = 'mosek';
end
opts = sdpsettings('solver', solver, 'verbose', 0);
sol = optimize([Fdecay, Fpositive], [], opts);
fprintf("YALMIP problem code: %d\n", sol.problem);
```

The final code is produced by the installed solver and should be interpreted with `sol.info`; it is not a package-owned status. The complete tested solver examples in sections 8.2 and 8.3 include assertions appropriate to their solver paths.

1.8 Generalize From An Interval To A Hypercube

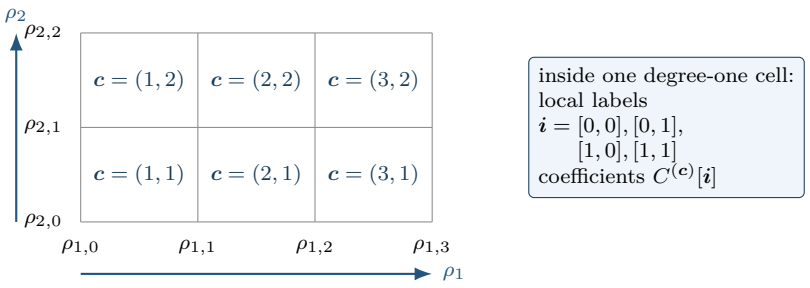
Different parameter directions need not have the same number of physical nodes. For example, four nodes in the first direction and three in the second create a 3-by-2 array of six physical cells:

MATLAB input

```
grid2 = {[-1 0 0.5 1], [10 15 20]};
G = pdbase(grid2, [1 1], 1);
fprintf("physical cells=%d, coefficients per cell=%d\n", ...
    G.ncell(), G.ncoeff());
disp(G.cells())
```

Output

```
physical cells=6, coefficients per cell=4
     1     1
     1     2
     2     1
     2     2
     3     1
     3     2
```



In ℓ dimensions, a physical node has an ℓ-component subscript $\boldsymbol{k} = (k_1, \ldots, k_\ell)$ and is written $\boldsymbol{\rho}_{\boldsymbol{k}}$. Direction s may have its own node count N_s. A physical cell has the separate identity $\boldsymbol{c} = (c_1, \ldots, c_\ell)$, so the number of cells is $\prod_s (N_s - 1)$. With the forward coordinates

$$\alpha_s = \frac{\rho_s - \rho_{s,c_s-1}}{\rho_{s,c_s} - \rho_{s,c_s-1}},$$

the local degree-(m) tensor representation is

$$M_{\boldsymbol{c}}(\boldsymbol{\alpha}) = \sum_{\boldsymbol{i}\in\{0,\ldots,m\}^{\ell}} \left(\prod_{s=1}^{\ell} B_{i_s}^{m}(\alpha_s) \right) C^{(\boldsymbol{c})}[\boldsymbol{i}].$$

The bracketed $\boldsymbol{i}$ is an ℓ-component local label, never a physical-node subscript. Earlier dimensions vary more slowly in `cells`, `lbls`, `coeffs`, and rate-vertex traversal. Cell-local spline and gridding constructions for PD-LMIs motivate this architecture [3, 4]; Appendix A develops the underlying basis from degree-one interpolation to the general tensor case.

2

Installation, Verification, And Lookup

2.1 Installation

Start MATLAB in the repository root, or set `projectRoot` to the absolute path of the checked-out repository. Add the package recursively, then remove the documentation directory from the executable path.

MATLAB input

```
projectRoot = "path/to/Differential Parameter—Dependent Linear Matrix Inequality";
addpath(genpath(projectRoot));
rmpath(genpath(fullfile(projectRoot, "doc")));
```

For local work from the repository root, this package also resolves from:

MATLAB input

```
addpath(pwd)
```

2.2 Verification

Run the current MATLAB test entry point from the repository root:

MATLAB input

```
results = tests.run_all();
```

The test entry point covers helper utilities, `pdbase`, `pdmat`, `pdvar`, and ordinary `pdlmi` behavior. The YALMIP-backed `pdvar` and `pdlmi` tests require YALMIP. Solver smoke tests additionally require an SDP solver.

2.3 Smoke Test

First confirm that MATLAB resolves the four renamed constructors from this repository and not from another toolbox:

MATLAB input

```
which pdbase —all
which pdmat —all
which pdvar —all
which pdlmi —all
```

Then exercise known-data inspection and the solver-facing assembly boundary:

MATLAB input

```
A = pdmat({[0 1]}, {1, 3}, Degree=1);
T = bernsteinTable(A, "oneLine");
disp(T)
```

Output

```
    CellSubscript      Expression
    -------------    ---------------

        {[1]}        "(1-alpha)*1 + alpha*3"
```

MATLAB input

```
yalmip('clear')
P = pdvar(2, {[0 1]}, "symmetric");
C = P >= 0;
F = toYalmip(C);
Cbox = C.applyFullBoxPreorder();
Fbox = toYalmip(Cbox);
```

`F` is the direct coefficient certificate; `Fbox` is the default full-box certificate for the same stored residual. Either can be passed to ordinary YALMIP calls, for example:

Call forms

```
sol = optimize(F, objective, opts);
```

2.4 Global Function Lookup

Category	Entries
Backend	`pdbase`, constructor, `cells`, `lbls`, `coeffs`, `ncell`, `ncoeff`, `npar`, `size`
Known data	`pdmat`, construction, `evaluate`, `bernsteinTable`, `disp`/`display`, `plot`, operators, matrix methods
Decision variables	`pdvar`, construction, `rhodiff`, affine/mixed algebra, matrix methods, comparisons
LMIs	`pdlmi`, construction, direct coefficient assembly, `applyPolya`, `applyFullBoxPreorder`, `toYalmip`, solver examples

3

pdbase Backend Reference

pdbase is the developer-facing backend parent for pdmat and pdvar. Ordinary users should model known data with pdmat and decision expressions with pdvar. This section documents pdbase because it explains the shared storage, traversal, and inspection contract used by both public modeling classes.

3.1 Lookup

Task	Function or field
Create backend object	pdbase
Inspect cells/labels	cells, lbls, coeffs
Count sizes	ncell, ncoeff, npar, size
Elevate stored coefficients	elevVals
Backend algebra mechanisms	elevLocalValues, bernElev, bernProd, mergeGrid
Read storage fields	GridInfo, LocalValues, metadata fields

3.2 pdbase Constructor

Syntax

Call forms

```
obj = pdbase(gridVectors, matrixSize, degree)
obj = pdbase(gridVectors, matrixSize, degree, localValues)
obj = pdbase(..., IsContinuous=tf)
obj = pdbase(..., ContainsDecision=tf)
obj = pdbase(..., HasRateDependence=tf)
obj = pdbase(..., RateBounds=rb)
obj = pdbase(..., SourceSummary=txt)
```

Arguments and options

- gridVectors is a cell vector with one strictly increasing node vector per parameter, such as {[0 1 2], [10 20]}.
- matrixSize is a positive two-element matrix size, such as [2 2].

- `degree` is a nonnegative integer Bernstein degree, such as `1`.
- `localValues` is the nested physical-cell storage. When omitted, `pdbase` creates a zero coefficient-backed object.
- `IsContinuous`, `ContainsDecision`, and `HasRateDependence` are logical metadata scalars. They default to false; subclasses set them to describe their own payload contract.
- `RateBounds` is empty or an $\ell \times 2$ matrix of finite lower and upper rate bounds, such as `RateBounds=[-1 1; -2 2]`. Nonempty bounds also mark the object as rate-dependent.
- `SourceSummary` is inspectable origin text and defaults to `"coefficient-backed"`.

Example

This backend example constructs a two-parameter parent object. The object has two physical cells because the first parameter has two intervals and the second parameter has one interval.

MATLAB input

```
obj = pdbase({[0 1 2], [10 20]}, [2 2], 1);
class(obj)
obj.MatrixSize
obj.Degree
obj.ncell()
obj.ncoeff()
obj.npar()
size(obj)
```

Output

```
ans =
    'pdbase'

ans =
     2     2

ans =
     1

ans =
     2

ans =
     4

ans =
     2

ans =
     2     2
```

The printed `ncoeff` value is $(1+1)^2 = 4$, one local Bernstein coefficient for each tensor-product label in the two parameter directions.

Malformed grid containers and node vectors raise `pdbase:InvalidGrid` and `pdbase:InvalidGridVector`. Invalid matrix sizes, degrees, or rate bounds raise `pdbase:InvalidMatrixSize`, `pdbase:InvalidDegree`, and `pdbase:InvalidRateBounds`. The nested tree must contain exactly one leaf per physical cell and $(m+1)^\ell$ coefficient columns per leaf; violations raise `pdbase:InvalidLocalValues` or `pdbase:InvalidCoefficientCell`. Payload matrices must have the declared size and be finite, real, and either numeric or supported real YALMIP expressions. Malformed payloads raise `pdbase:InvalidCoefficientPayload`. Rate-affine payload structures and `HasRateDependence=true` require compatible nonempty `RateBounds`.

3.3 Storage Fields

Field	Meaning
`GridInfo`	Grid vectors, node counts, and grid bounds.
`MatrixSize`	Size of each matrix coefficient or matrix expression.
`Degree`	Local Bernstein degree stored in every physical cell.
`LocalValues`	Nested cell tree indexed by physical-cell subscript; each leaf stores coefficient cells.
`IsContinuous`	Metadata flag; subclasses are responsible for boundary sharing if continuity is true.
`ContainsDecision`	True when coefficient payloads include YALMIP decision variables.
`HasRateDependence`	True when the object carries rate metadata or rate-vertex rows.
`RateBounds`	Parameter-rate lower and upper bounds used by `rhodiff` and `pdlmi`.
`SourceSummary`	Short origin label such as `coefficient-backed`, `decision`, or `derivative`.

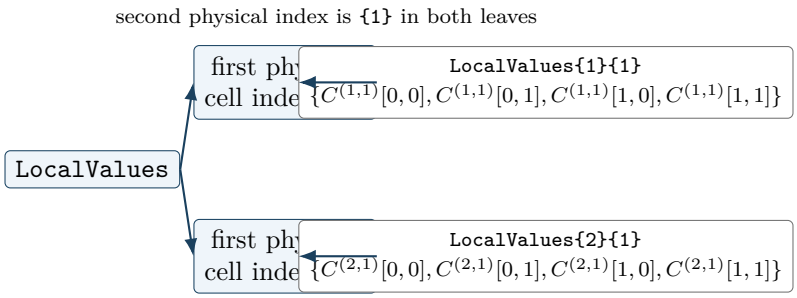


Figure 1: Nested cell-local storage for a two-parameter, degree-one object on `{[0 1 2], [10 20]}`. There are two physical cells, $[1,1]$ and $[2,1]$. Each leaf is a flat local coefficient cell in `lbls` order: $[0,0]$, $[0,1]$, $[1,0]$, $[1,1]$.

The nesting follows physical-cell subscripts, not the Bernstein labels. Start with `LocalValues\{i1\}`, then take one cell index per remaining parameter; this selects physical cell $[i_1, i_2, \ldots, i_\ell]$. Only after that selection does the leaf's flat cell array enumerate the $(m+1)^\ell$ local Bernstein coefficients. Thus `coeffs(obj, [2 1])` returns the lower leaf in fig. 1, while `lbls(obj)` explains the order inside that leaf.

3.4 Inspection: `cells`, `lbls`, And `coeffs`

Syntax

Call forms

```
subs = cells(obj)
subs = obj.cells()

labels = lbls(obj)
labels = obj.lbls()

c = coeffs(obj, cellSubs)
c = obj.coeffs(cellSubs)
```

Arguments and options

- `obj` is a `pdbase` object or a `pdmat`/`pdvar` subclass instance using the shared cell-local storage convention.
- `cellSubs` selects one physical cell for `coeffs`. Supply a numeric row vector such as `[2 1]` or a cell array of scalar subscripts such as `{2,1}`.
- `cells` returns physical hypercube subscripts in the shared traversal order; `lbls` returns local Bernstein labels in the shared coefficient order.
- `c` is the flat coefficient cell array for the selected physical cell, ordered consistently with `lbls(obj)`.

Example

The label order is shared by `pdmat`, `pdvar`, and `pdlmi`. In this two-parameter degree-one object, each physical cell stores four coefficient entries with labels $[0,0]$, $[0,1]$, $[1,0]$, and $[1,1]$.

MATLAB input

```
obj = pdbase({[0 1 2], [10 20]}, [1 1], 1);
lbls(obj)
cells(obj)
c = coeffs(obj, [2 1]);
numel(c)
```

Output

```
ans =
     0     0
     0     1
     1     0
     1     1

ans =
     1     1
     2     1

ans =
     4
```

3.5 Counts And Shape

Syntax

Call forms

```
n = ncell(obj)
n = obj.ncell()

n = ncoeff(obj)
n = obj.ncoeff()

n = npar(obj)
n = obj.npar()

sz = size(obj)
d = size(obj, dim)
```

Arguments and options

- `obj` is a `pdbase` object or subclass instance.
- `dim` is a positive integer matrix dimension for `size`; it follows MATLAB's ordinary `size(obj,dim)` convention.
- `ncell` returns $\prod_r (N_r - 1)$, the number of physical hypercube cells.
- `ncoeff` returns $(m+1)^\ell$, the number of local Bernstein coefficients stored per physical cell.
- `npar` returns the parameter dimension ℓ. `size` returns the matrix payload size, never the physical grid size.

Example

For a two-parameter degree-one backend object, the first grid has two physical intervals and the second has one. The matrix payload is 2-by-3, independently of the grid shape.

MATLAB input

```
obj = pdbase({[0 1 2], [10 20]}, [2 3], 1);
fprintf('ncell = %d\n', obj.ncell());
fprintf('ncoeff = %d\n', obj.ncoeff());
fprintf('npar = %d\n', obj.npar());
disp(size(obj))
disp(size(obj, 1))
disp(size(obj, 2))
disp(size(obj, 3))
```

Output

```
ncell = 2
ncoeff = 4
npar = 2
     2     3

     2

     3

     1
```

3.5.1 `elevVals`

Syntax

Call forms

```
vals = obj.elevVals(degreeIncrement)
```

Arguments and options

- `degreeIncrement` is a finite nonnegative integer scalar. It is added to `obj.Degree` in every parameter direction; zero returns coefficient evidence at the current degree.

Output

`vals` is a nested `LocalValues`-shaped cell tree. Each physical cell contains coefficients of degree `obj.Degree + degreeIncrement`, with the represented polynomial, physical-cell tree, and any rate-row ordering preserved. The method does not modify `obj` or its stored coefficient evidence.

Validation and errors

An invalid increment raises `pdbase:InvalidDegreeIncrement`. A function-only `pdmat` has no stored Bernstein coefficient evidence, so this method raises `pdbase:MissingCoefficientEvidence`.

Relationship to backend and public algebra. Unlike protected `bernElev`, which elevates one flat cell coefficient family for internal alignment, `elevVals` is a public whole-object query returning elevated `LocalValues`. Public coefficient algebra uses `bernElev` internally when compatible operands need degree alignment; it does not mutate either operand.

3.5.2 `elevLocalValues`

Internal/protected mechanism. This method elevates a temporary nested coefficient tree after public algebra has remapped operands to a common physical grid. It is documented as backend context, not as a supported user call form.

Syntax

Call forms

```
vals = elevLocalValues(obj, vals, fromDeg, toDeg)
vals = elevLocalValues(obj, vals, fromDeg, toDeg, grid)
```

Each leaf must be a cell array with one column per degree-`fromDeg` Bernstein label. The method elevates each rate-vertex row independently to `toDeg`; it never mixes rows. The optional `grid` supplies the temporary physical-cell layout, and otherwise defaults to `obj.GridInfo.Vectors`. Equal source and target degrees return the input tree unchanged. A malformed leaf raises `pdbase:InvalidCoefficientCell`. Public users encounter this behavior through grid merging, assignment, concatenation, and arithmetic rather than by calling the protected method directly.

3.5.3 `bernElev`

Internal/protected mechanism. This signature documents the backend algorithm used by public algebra; it is not a supported user call form.

Syntax

Call forms

```
out = bernElev(obj, coeffs, fromDeg, toDeg)
```

Arguments and options

- `coeffs` is a flat coefficient-cell family in the same label order as `lbls(obj)`.
- `fromDeg` and `toDeg` are nonnegative integer degrees, with `toDeg >= fromDeg`.
- `out` contains $(\texttt{toDeg}+1)^{\ell}$ elevated coefficients in the same label order.

Example

MATLAB input

```
A = pdmat({[0 1]}, {[1 2]}, Degree=1);
B = pdmat({[0 1]}, {1, 2, 3}, Degree=2);
C = A + B;
C.Degree
```

Output

```
ans =
     2
```

The example invokes the protected helper through public addition; users do not call `bernElev` directly.

3.5.4 `bernProd`

Internal/protected mechanism. This signature documents the backend algorithm used by public matrix multiplication; it is not a supported user call form.

Syntax

Call forms

```
out = bernProd(obj, lhs, lhsDeg, rhs, rhsDeg)
```

Arguments and options

- `lhs` and `rhs` are flat coefficient-cell families whose matrix payload inner dimensions are compatible for multiplication.
- `lhsDeg` and `rhsDeg` are their nonnegative local Bernstein degrees.
- `out` is the binomial-normalized coefficient convolution at degree `lhsDeg + rhsDeg`; payload products follow MATLAB matrix multiplication rules.

Example

MATLAB input

```
A = pdmat({[0 1]}, {[1 2]}, Degree=1);
P = A * A;
P.Degree
```

Output

```
ans =
     2
```

The example invokes the protected helper through public matrix multiplication; users do not call `bernProd` directly.

3.5.5 `mergeGrid`

Internal/protected mechanism. This signature documents the backend grid-refinement algorithm used by public algebra; it is not a supported user call form.

Syntax

Call forms

```
grid = obj.mergeGrid(errId, other1, other2, ...)
```

Arguments and options

- `errId` is the error identifier used when parameter counts or physical bounds are incompatible.
- `other1, other2, ...` are coefficient-backed operands, including function-backed operands constructed with an explicit `Degree`. They must have equal parameter counts and matching first and last nodes on every parameter axis.

- `grid` has the sorted union of the interior nodes. Public algebra re-expresses operands on these cells before degree elevation, coefficient-wise addition, or product convolution.

This protected helper is invoked by public algebra methods rather than called directly by users.

Example

MATLAB input

```
A = pdmat({[0 1]}, {1, 2}, Degree=1);
B = pdmat({[0 0.5 1]}, {10, 20, 30}, Degree=1);
C = A + B;
C.GridInfo.Vectors{1}
```

Output

```
ans =
         0    0.5000    1.0000
```

The example invokes the protected helper through public addition; users do not call `mergeGrid` directly.

3.6 See Also

`pdmat`, `pdvar`, `cells`, `coeffs`, `lbls`, `ncell`, `ncoeff`, `npar`, `size`, `elevVals`, `elevLocalValues`, `bernElev`, `bernProd`, `mergeGrid`.

4

pdmat Reference

pdmat stores known finite real matrix data on a tensor grid. Objects can be coefficient-backed, function-only, or function-backed with explicit Bernstein coefficient evidence. Coefficient-backed objects participate in algebra. Function-only objects evaluate exactly through the stored handle, but do not expose coefficient evidence for bernsteinTable or coefficient algebra.

4.1 Lookup

Task	Entry
Construct data	pdmat
Evaluate	evaluate
Inspect/display	bernsteinTable, disp, display
Plot	plot
Arithmetic	+, -, unary +/-, *
Matrix/index methods	shape and structural methods, indexing and assignment

4.2 Construction

Syntax

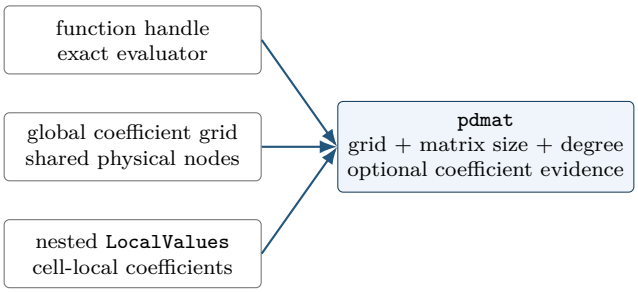
Call forms

```
A = pdmat(gridVector, source)
A = pdmat(gridVector, source, Degree=m)
A = pdmat(gridVectors, source)
A = pdmat(gridVectors, source, Degree=m)
```

Arguments and options

- gridVector is a numeric vector used as a one-parameter grid shorthand, such as [0 1 2].
- gridVectors is a cell vector with one numeric grid vector per parameter, such as {[0 1], [10 20]}.
- source may be a function handle, a global cell grid of numeric Bernstein coefficients, or explicit nested LocalValues; for example, {1, 2, 3} is scalar degree-two data on one cell.

- `Degree=m` sets the local Bernstein degree. For function handles, omitting `Degree` creates a function-only object; passing a value such as `Degree=2` validates and stores Bernstein coefficient evidence while retaining the exact function handle for `evaluate`.



Example

The scalar-grid shorthand is the shortest coefficient-backed form. The three numeric cells below are the global Bernstein coefficients for two physical cells with degree one.

MATLAB input

```
A = pdmat([0 1 2], {10, 20, 30}, Degree=1)
size(A)
A.Degree
A.SourceSummary
```

Output

```
A =
  pdmat [1 1] over 1-D grid, degree 1, source coefficient-backed

ans =
     1     1

ans =
     1

ans =
    "coefficient-backed"
```

For tensor grids, pass one grid vector per parameter. The source cell array matches the global coefficient grid implied by the two parameter directions.

MATLAB input

```
B = pdmat({[0 1], [10 20]}, {1 2; 3 4}, Degree=1)
B.GridInfo.Vectors{2}
B.lbls()
```

Output

```
B =
  pdmat [1 1] over 2-D grid, degree 1, source coefficient-backed

ans =
    10    20

ans =
     0     0
     0     1
     1     0
     1     1
```

Use explicit nested `LocalValues` when each physical cell must be written directly. Here each cell stores two 1×2 row-vector coefficients.

MATLAB input

```
localVals = {{[1 0], [0 1]}, {[2 0], [0 2]}};
C = pdmat({[0 1 2]}, localVals, Degree=1)
size(C)
C.coeffs(2)
```

Output

```
C =
  pdmat [1 2] over 1-D grid, degree 1, source coefficient-backed

ans =
     1     2

ans =
  1x2 cell array
    {[2 0]}    {[0 2]}
```

When `source` is a function handle and `Degree` is omitted, the object keeps the exact evaluator but does not claim coefficient evidence.

MATLAB input

```
F = pdmat({[0 1]}, @(rho) [rho rho^2])
F.SourceSummary
F.evaluate(0.5)
```

Output

```
F =
  pdmat [1 2] over 1-D grid, degree 1, source function

ans =
    "function"

ans =
    0.5000    0.2500
```

Add `Degree` for a polynomial function when later coefficient inspection or algebra should use Bernstein evidence. The exact function handle is still used for point evaluation.

MATLAB input

```
G = pdmat({[0 1]}, @(rho) rho.^2, Degree=2)
G.SourceSummary
G.coeffs(1)
```

Output

```
G =
  pdmat [1 1] over 1-D grid, degree 2, source function-bernstein

ans =
    "function-bernstein"

ans =
  1x3 cell array
    {[0]}    {[0]}    {[1]}
```

These forms all create finite known-data objects, but only the coefficient- backed and function-Bernstein forms can participate in coefficient algebra.

Public property

`FunctionHandle` retains the supplied function handle for function-only and function-Bernstein sources; it is empty for cell-backed sources. Its set access is private.

Validation and errors

Constructor options must be name–value pairs. Malformed syntax raises `pdmat:InvalidOptions`; backend-only options raise `pdmat:UnsupportedOption`; unknown names raise `pdmat:UnknownOption`. Grid errors use `pdmat:InvalidGrid` or `pdmat:InvalidGridVector`.

An unsupported source kind raises `pdmat:InvalidSource`. Function arity or initial-call failures raise `pdmat:InvalidFunctionHandle`; empty, complex, or otherwise invalid initial outputs raise `pdmat:InvalidFunctionOutput`. Global coefficient grids with inconsistent finite-real payload sizes raise `pdmat:InvalidData`. Incompatible inferred or explicit degrees raise `pdmat:InvalidDegree`; malformed nested cell storage raises `pdmat:InvalidLocalValues`. Malformed explicit coefficient-leaf counts or shapes raise `pdmat:InvalidCoefficientCell`; invalid, nonfinite, nonreal, or wrong-size coefficient payloads raise `pdmat:InvalidCoefficientPayload`. With an explicit `Degree`, a function that is not representable at that degree raises `pdmat:NonBernsteinPolynomial`; a failed or size-inconsistent validation probe raises `pdmat:InvalidFunctionValue`. Nested local data with mismatched shared faces is retained as discontinuous data and emits warning `pdmat:DiscontinuousLocalValues`.

4.3 `evaluate`

Syntax

Call forms

```
val = evaluate(A, point)
val = A.evaluate(point)
```

Arguments and options

- `A` is a `pdmat` object.
- `point` is a finite real scalar for a one-parameter object, such as `0.25`, or a finite real row vector with one entry per parameter, such as `[0.25 12]`. Every entry must lie within the corresponding grid bounds.
- `val` is a numeric matrix with `size(A)`. Coefficient-backed objects use local Bernstein interpolation; function-backed objects call their stored function handle.

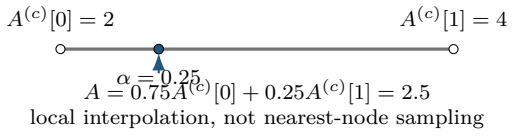
Validation and errors

A point with the wrong type, length, or finite-real shape raises `pdmat:InvalidPoint`; a point outside any grid bound raises `pdmat:PointOutOfBounds`. If a stored function fails or returns a nonfinite, complex, empty, or size-inconsistent matrix at the requested point, `evaluate` raises `pdmat:InvalidFunctionValue`.

For a degree-one scalar coefficient-backed object on $[\rho_0, \rho_1]$,

$$A(\rho) = (1-\alpha)A^{(c)}[0] + \alpha A^{(c)}[1], \qquad \alpha = \frac{\rho - \rho_0}{\rho_1 - \rho_0}.$$

The same local-coordinate rule is applied entrywise for matrix-valued data.



Example

Use the function-call form when the object is naturally one input among several MATLAB expressions.

MATLAB input

```
A = pdmat({[0 1]}, {2, 4}, Degree=1);
val = evaluate(A, 0.25)
```

Output

```
val =
    2.5000
```

At $\rho = 0.25$, the public local coordinate is $\alpha = 0.25$, so the interpolated value is $0.75 \cdot 2 + 0.25 \cdot 4 = 2.5$. The dot-call form is equivalent and can be convenient in command-window exploration.

MATLAB input

```
A = pdmat({[0 1]}, {2, 4}, Degree=1);
val = A.evaluate(0.75)
```

Output

```
val =
    3.5000
```

Function-backed objects route through the retained function handle, so non-polynomial expressions can still be evaluated exactly at a point.

MATLAB input

```
F = pdmat({[0 pi]}, @(rho) sin(rho));
val = F.evaluate(pi/2)
```

Output

```
val =
     1
```

4.4 Visualization And Inspection

4.4.1 `bernsteinTable`

Syntax

Call forms

```
T = bernsteinTable(A)
T = bernsteinTable(A, cellSubs)
T = bernsteinTable(A, "oneLine")
T = bernsteinTable(A, cellSubs, "oneLine")
```

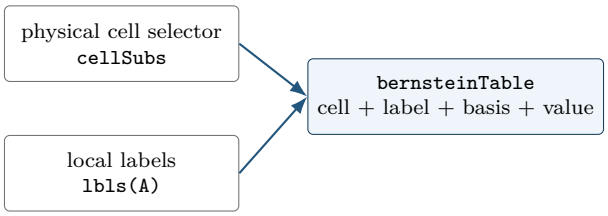
Arguments and options

- `cellSubs` selects one physical cell.
- `"oneLine"` returns one row per selected physical cell with a compact Bernstein expression.
- Without `"oneLine"`, the full table uses seven metadata columns; the output below shows their exact MATLAB names.

For one parameter, the `Basis` column reports

$$\binom{m}{j}(1-\alpha)^{m-j}\alpha^{j}.$$

For multiple parameters, the basis is the tensor product of that expression in each local coordinate, using `alpha1`, `alpha2`, and so on. `LocalIndex` is the local Bernstein label. `CoeffSubscript` maps the local label back to the global coefficient subscript on the physical grid. `IsPhysicalNode` is true when that coefficient lies on a physical grid node.



The table reports stored coefficient evidence. It does not sample a function and does not create solver constraints.

Example

MATLAB input

```
A = pdmat({[0 1]}, {[0 1], [1 2]}, Degree=1);
T = bernsteinTable(A, 1, "oneLine");
disp(T)
```

Output

```
    CellSubscript          Expression
    -------------    ------------------------

        {[1]}        "(1-alpha)*[0 1] + alpha*[1 2]"
```

Use the full table when the metadata columns are the point of the example:

MATLAB input

```
A = pdmat({[0 1]}, {1, 2, 3}, Degree=2);
T = bernsteinTable(A);
disp(T)
```

Output

```
    TermIndex    CellSubscript    CoeffSubscript    LocalIndex      Basis      IsPhysicalNode    Value
    ---------    -------------    --------------    ----------    ---------    --------------    -----

        1            {[1]}            {[1]}           {[0]}       "(1-alpha)^2"       true          {[1]}
        2            {[1]}            {[2]}           {[1]}       "2(1-alpha)alpha"   false         {[2]}
        3            {[1]}            {[3]}           {[2]}       "alpha^2"           true          {[3]}
```

Validation and errors

A function-only object has no coefficient evidence and raises `pdmat:FunctionOnlyBernsteinTable`. Unknown text options, repeated selectors, or invalid physical-cell selectors raise `pdmat:InvalidBernsteinTableInput`.

4.4.2 `disp` And `display`

Syntax

Call forms

```
disp(A)
display(A)
A
```

Arguments and options

`disp(A)` prints a compact one-line summary. `display(A)` is the command-window display used when an expression is not terminated by a semicolon; it prints grid, degree, coefficient, and source metadata.

Example

MATLAB input

```
A = pdmat({[0 1]}, {1, 2}, Degree=1);
disp(A)
display(A)
```

Output

```
pdmat [1 1] over 1-D grid, degree 1, source coefficient-backed
A =
  pdmat with 1-by-1 matrix values
  Parameters: 1
    rho_1: [0, 1], 2 nodes
  Degree: 1
  Physical cells: 1
  Coefficients per cell: 2
  Continuous: true
  Source: coefficient-backed
  Function handle: false
```

4.4.3 `plot`

Syntax

Call forms

```
h = plot(A)
h = plot(A, dims)
h = plot(A, SamplesPerCell=n)
h = plot(A, dims, SamplesPerCell=n)
h = plot(A, ..., Name, Value)
```

Arguments and options

- `dims` is one or two parameter indices. For one-parameter objects, the default is `1`; otherwise the default is `[1 2]`, as in `plot(A, [1 2])`.
- `SamplesPerCell` is the package-owned sampling option. The default is `15`; use `SamplesPerCell=4` for a coarse diagnostic plot.
- Remaining name-value pairs pass through to MATLAB graphics. One- dimensional plots pass them to `plot`; two-dimensional plots pass them to `surf`. Examples include `LineWidth`, `FaceAlpha`, and `EdgeColor`.
- The output `h` is the vector of graphics handles, one entry per matrix element.

Validation and errors

An invalid parameter index, a repeated dimension, or a request for more than two plotting dimensions raises `pdmat:InvalidPlotDimensions`. Malformed plotting options or a nonpositive integer `SamplesPerCell` raise `pdmat:InvalidPlotOptions`. Errors from MATLAB graphics options remain MATLAB graphics errors.

Example

The one-dimensional call returns one graphics handle per matrix entry. The example inspects the generated sample count and line styling without relying on an interactive figure window.

MATLAB input

```
fig = figure('Visible','off');
A = pdmat({[0 1]}, @(rho) [rho, rho^2]);
h = plot(A, SamplesPerCell=4, LineWidth=2);
numel(h)
numel(h(1).XData)
h(1).LineWidth
h(1).YData([1 end])
close(fig)
```

Output

```
ans =
     2

ans =
     5

ans =
     2

ans =
     0     1
```

With four samples per cell on one physical interval, MATLAB stores five plotted node values including both endpoints.

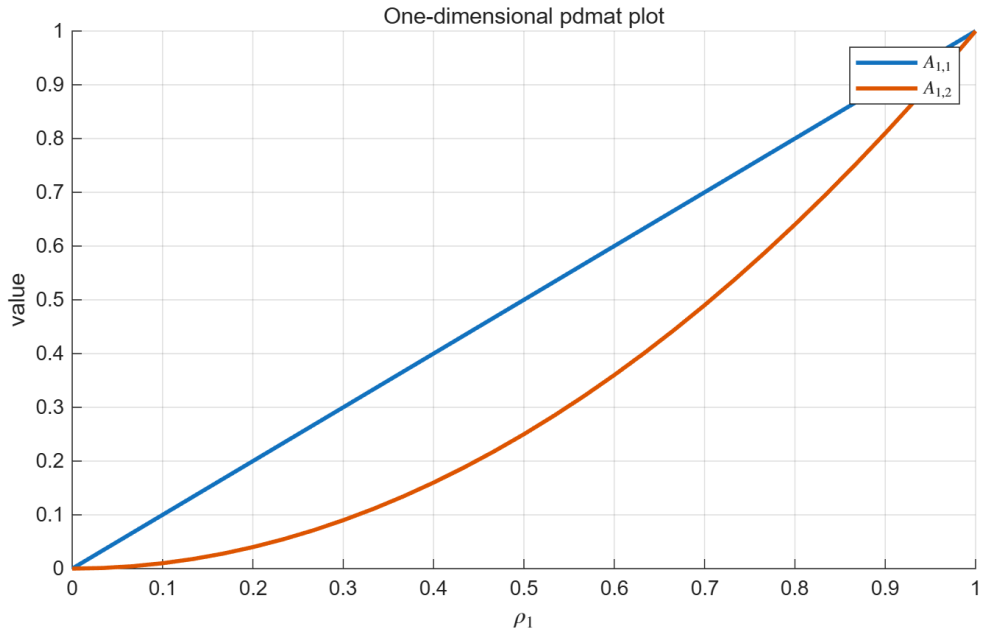


Figure 2: One-dimensional `pdmat/plot` output generated from MATLAB.

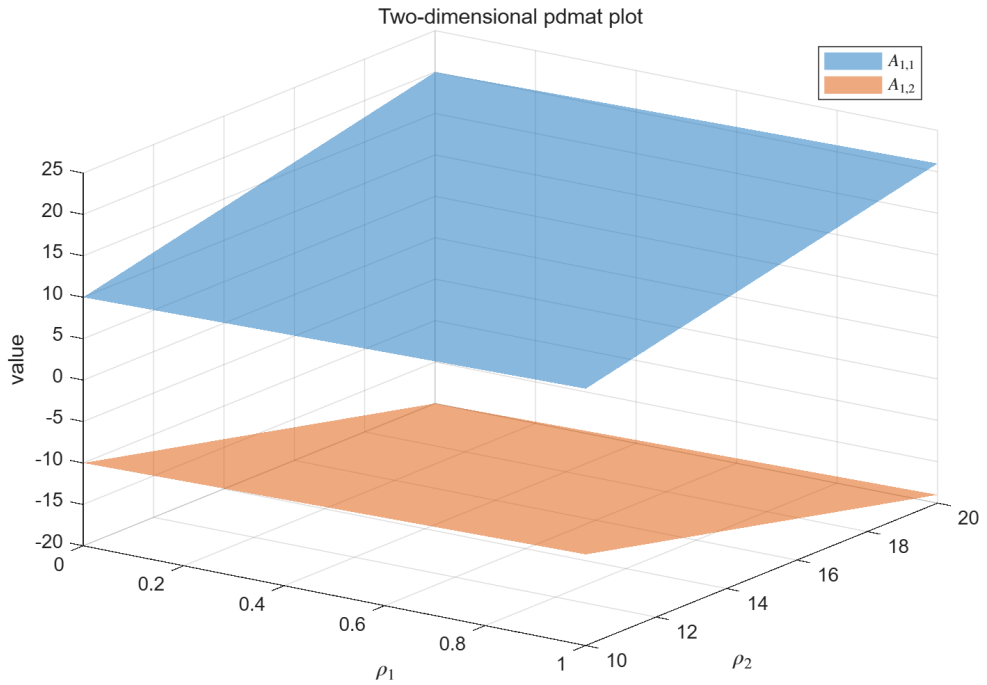


Figure 3: Two-dimensional `pdmat/plot` surface output generated from MATLAB.

4.5 Algebra And Matrix Operations

A `pdmat` operation acts on complete matrix coefficients while the physical grid remains attached to the result. The blocks below deliberately show one operation family at a time: purpose, supported syntax, input, immediate output, and—when the result is spatial—a before/after picture.

4.5.1 Signs, Addition, And Subtraction

Use signs, sums, and differences to build known matrix expressions. Compatible physical bounds are refined to the union grid, and lower-degree operands are elevated exactly before matching coefficients are combined. The corresponding MATLAB overload names are `plus`, `minus`, `uplus`, and `uminus`.

Syntax

Call forms

```
S = A + B
S = A + M
S = M + A
D = A - B
D = A - M
D = M - A
P = +A
N = -A
```

Here `M` is a finite real scalar or size-compatible matrix. A scalar broadcasts to the matrix payload size. The first example makes the degree alignment visible.

MATLAB input

```
A = pdmat({[0 1]}, {1, 2}, Degree=1);
B = pdmat({[0 1]}, {10, 20, 30}, Degree=2);
S = A + B;
cS = S.coeffs(1);
fprintf("sum degree=%d\n", S.Degree);
disp([cS{:}])
```

Output

```
sum degree=2
   11.0000   21.5000   32.0000
```

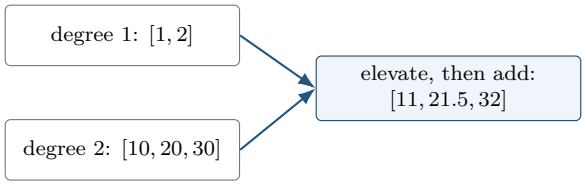
The middle coefficient of the elevated A is $(1+2)/2 = 1.5$, so the middle sum is $1.5 + 20 = 21.5$. Subtraction and unary signs act immediately on the same local coefficients.

MATLAB input

```
D = 5 - A;
N = -A;
cD = D.coeffs(1);
fprintf("5-A coefficients: "); disp([cD{:}])
fprintf("-A at rho=1: %.0f\n", N.evaluate(1));
fprintf("class(+A) = %s\n", class(+A));
```

Output

```
5-A coefficients:      4     3
-A at rho=1: -2
class(+A) = pdmat
```



When grids have the same outer bounds but different interior nodes, the union grid is used before

addition or subtraction. For example, `pdmat({[0 1]},...)+pdmat({[0 .5 1]},...)` has grid `[0 .5 1]`; each operand is represented exactly on both resulting cells.

4.5.2 Ordered Matrix Multiplication

Use `mtimes` for a matrix product. The inner payload dimensions must match. Two parameter-dependent factors produce the degree sum; a numeric factor is constant and does not raise the degree.

Syntax

Call forms

```
K = A * B
K = A * M
K = M * A
```

For a fixed physical cell (c), let $A = \sum_i B_i^m A^{(c)}[i]$ and $D = \sum_j B_j^n D^{(c)}[j]$. Then (K=AD) has coefficients

$$K^{(c)}[k] = \sum_{i+j=k} \frac{\binom{m}{i}\binom{n}{j}}{\binom{m+n}{k}} A^{(c)}[i] D^{(c)}[j].$$

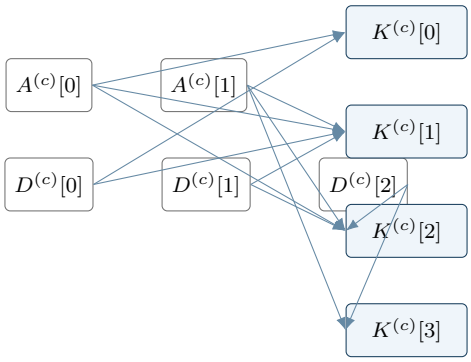
The left/right matrix order is never exchanged.

MATLAB input

```
A = pdmat({[0 1]}, {1, 2}, Degree=1);
B = pdmat({[0 1]}, {10, 20, 30}, Degree=2);
K = A * B;
cK = K.coeffs(1);
fprintf("product degree=%d\n", K.Degree);
disp([cK{:}])
```

Output

```
product degree=3
   10.0000   20.0000   36.6667   60.0000
```



Coefficient labels add under the weighted convolution; payload matrices multiply in the written order.

For tensor data, label addition and binomial weights are applied independently in each parameter direction. Two degree-one factors in two parameters produce degree two and $3^2 = 9$ coefficients per physical cell.

Validation and errors

Incompatible operands for `+`, `-`, and `*` raise `pdmat:InvalidAddition`, `pdmat:InvalidSubtraction`, or `pdmat:InvalidMultiplication`. Parameter counts or outer bounds that cannot be merged raise `pdmat:MixedGrid`. Coefficient algebra on a function-only object raises `pdmat:FunctionOnlyAlgebra`; construct the function with an explicit verified `Degree` when coefficient evidence is required.

4.6 Matrix Methods

These methods transform every local matrix coefficient in the same way. They do not transpose, reshape, or concatenate the physical grid. Except for the documented evaluator/display/shape paths, structural methods require coefficient evidence.

4.6.1 Shape, Equality, And Display Protocols

Shape queries describe the matrix payload, not the grid. Equality compares coefficient-backed functions after compatible grid refinement and degree elevation. The protocol methods support ordinary MATLAB indexing syntax and dot access; they are not a second storage API.

Syntax

Call forms

```
sz = size(A)
d = size(A, dim)
n = height(A)
n = width(A)
n = length(A)
n = numel(A)
n = ndims(A)
tf = isequal(A, B, ...)
disp(A)
display(A)
idx = end(A, k, n)
n = numArgumentsFromSubscript(A, S, ctx)
```

MATLAB input

```
A = pdmat({[0 1]}, ...
    {[1 2 3; 4 5 6], [10 20 30; 40 50 60]}, Degree=1);
fprintf("size=%d-by-%d, height=%d, width=%d\n", ...
    size(A,1), size(A,2), height(A), width(A));
fprintf("length=%d, numel=%d, ndims=%d, equal(+A)=%d\n", ...
    length(A), numel(A), ndims(A), isequal(A,+A));
disp(A)
```

Output

```
size=2-by-3, height=2, width=3
length=3, numel=6, ndims=2, equal(+A)=1
pdmat [2 3] over 1-D grid, degree 1, source coefficient-backed
```

A function-only object can use shape queries and compares its stored metadata and function handle. A coefficient-backed equality that cannot form a common comparison grid raises `pdmat:InvalidEquality`. The detailed multi-line `display` transcript is shown in section 4.4.2.

4.6.2 Transpose And Conjugate Transpose

Both transpose forms act on every coefficient. Because `pdmat` payloads are finite and real, conjugation does not change their values.

Syntax

Call forms

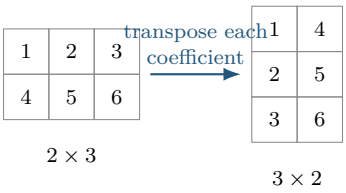
```
T = transpose(A)
H = ctranspose(A)
T = A.'
H = A'
```

MATLAB input

```
A = pdmat({[0 1]}, ...
    {[1 2 3; 4 5 6], [10 20 30; 40 50 60]}, Degree=1);
T = A.';
H = A';
fprintf("transpose size=%d-by-%d, equal forms=%d\n", ...
    size(T,1), size(T,2), isequal(T,H));
disp(T.evaluate(0))
```

Output

```
transpose size=3-by-2, equal forms=1
     1     4
     2     5
     3     6
```



4.6.3 Vectorization, Reshape, And Squeeze

Use these methods to change the matrix view while preserving column-major element order, the parameter grid, the degree, and every coefficient value. `reshape` accepts exactly two positive dimensions with at most one inferred `[]` dimension. `squeeze` retains the package's two-dimensional matrix convention.

Syntax

Call forms

```
V = vec(A)
R = reshape(A, [m n])
R = reshape(A, m, n)
S = squeeze(A)
```

MATLAB input

```
A = pdmat({[0 1]}, ...
    {[1 2 3; 4 5 6], [10 20 30; 40 50 60]}, Degree=1);
V = vec(A);
R = reshape(A, [3 2]);
S = squeeze(A);
fprintf("vec=%d-by-%d, reshape=%d-by-%d, squeeze=%d-by-%d\n", ...
    size(V,1), size(V,2), size(R,1), size(R,2), size(S,1), size(S,2));
v0 = V.evaluate(0);
disp(v0.')
```

Output

```
vec=6-by-1, reshape=3-by-2, squeeze=2-by-3
     1     4     2     5     3     6
```

Malformed or element-count-changing reshape requests raise `pdmat:InvalidReshape`.

4.6.4 Diagonals And Trace

`diag` extracts a selected diagonal from a matrix, or builds a square diagonal matrix from a vector. A finite integer offset `k` may not select an empty diagonal. `trace` returns a 1-by-1 `pdmat`.

Syntax

Call forms

```
d = diag(A)
d = diag(A, k)
D = diag(v)
D = diag(v, k)
t = trace(A)
```

MATLAB input

```
A = pdmat({[0 1]}, {[1 2; 3 4], [10 20; 30 40]}, Degree=1);
d = diag(A);
D = diag(d);
t = trace(A);
disp(d.evaluate(0))
disp(D.evaluate(0))
fprintf("trace at rho=0: %.0f\n", t.evaluate(0));
```

Output

```
     1
     4

     1     0
     0     4
trace at rho=0: 5
```

Invalid offsets or empty selections raise `pdmat:InvalidDiag`.

4.6.5 Triangular Parts And Reductions

Triangular extraction preserves or zeros entries relative to a selected diagonal. Reductions act along matrix dimensions; `"all"` is supported by `sum` and `mean`.

Syntax

Call forms

```
L = tril(A)
L = tril(A, k)
U = triu(A)
U = triu(A, k)
S = sum(A)
S = sum(A, dim)
S = sum(A, "all")
M = mean(A)
M = mean(A, dim)
M = mean(A, "all")
```

```
C = cumsum(A)
C = cumsum(A, dim)
```

First inspect the two triangular results.

MATLAB input

```
A = pdmat({[0 1]}, {[1 2; 3 4], [10 20; 30 40]}, Degree=1);
L = tril(A);
U = triu(A);
disp(L.evaluate(0))
disp(U.evaluate(0))
```

Output

```
    1    0
    3    4

    1    2
    0    4
```

Then apply reductions to the same coefficient matrix.

MATLAB input

```
rowSum = sum(A, 2);
allMean = mean(A, "all");
running = cumsum(A, 2);
disp(rowSum.evaluate(0))
fprintf("mean(all) at rho=0: %.1f\n", allMean.evaluate(0));
disp(running.evaluate(0))
```

Output

```
    3
    7
mean(all) at rho=0: 2.5
    1    3
    3    7
```

Invalid triangular offsets raise `pdmat:InvalidTriangularPart`. Invalid reduction calls raise `pdmat:InvalidSum`, `pdmat:InvalidMean`, or `pdmat:InvalidCumsum`.

4.6.6 Reordering And Rotation

These methods reorder rows or columns inside every coefficient; they never reverse the scheduling grid.

Syntax

Call forms

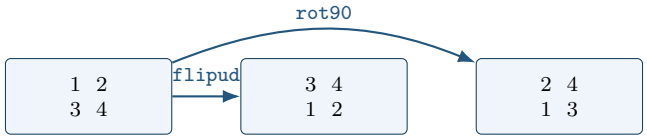
```
B = flip(A)
B = flip(A, dim)
B = flipud(A)
B = fliplr(A)
B = rot90(A)
B = rot90(A, k)
```

MATLAB input

```
A = pdmat({[0 1]}, {[1 2; 3 4], [10 20; 30 40]}, Degree=1);
F = flipud(A);
R = rot90(A);
disp(F.evaluate(0))
disp(R.evaluate(0))
```

Output

```
    3    4
    1    2

    2    4
    1    3
```



Invalid dimensions or quarter-turn counts raise `pdmat:InvalidFlip` or `pdmat:InvalidRot90`.

4.6.7 Concatenation And Block Assembly

Concatenation joins payload matrices after compatible grid refinement and degree elevation. Numeric blocks are promoted to constant known data. `cat` supports only dimensions 1 and 2.

Syntax

Call forms

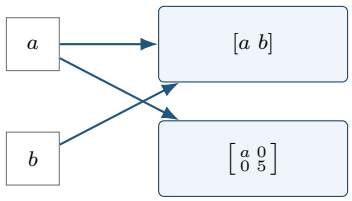
```
H = horzcat(A, B, ...)
V = vertcat(A, B, ...)
H = [A, B]
V = [A; B]
C = cat(dim, A, B, ...)
D = blkdiag(A, B, ...)
```

MATLAB input

```
a = pdmat({[0 1]}, {1, 2}, Degree=1);
b = pdmat({[0 1]}, {10, 20}, Degree=1);
H = [a, b];
V = [a; b];
D = blkdiag(a, 5);
disp(H.evaluate(0))
disp(V.evaluate(0))
disp(D.evaluate(0))
```

Output

```
     1    10

     1
    10

     1     0
     0     5
```



Incompatible blocks or syntax raise `pdmat:InvalidConcatenation`; unsupported dimensions raise `pdmat:UnsupportedCatDimension`; invalid block-diagonal inputs raise `pdmat:InvalidBlkdiag`. Function-only operands raise `pdmat:FunctionOnlyAlgebra`.

4.6.8 Repetition

`repmat` tiles the matrix payload. It accepts a positive scalar `m`, interpreted as `[m m]`, two positive counts, or a two-element count vector. Grid and Bernstein degree remain unchanged.

Syntax

Call forms

```
R = repmat(A, m)
R = repmat(A, m, n)
R = repmat(A, [m n])
```

MATLAB input

```
v = pdmat({[0 1]}, {[1 2], [3 4]}, Degree=1);
R = repmat(v, [2 2]);
fprintf("repeated size=%d-by-%d, degree=%d\n", ...
    size(R,1), size(R,2), R.Degree);
disp(R.evaluate(0))
```

Output

```
repeated size=2-by-4, degree=1
     1     2     1     2
     1     2     1     2
```

Invalid repetition shapes raise `pdmat:InvalidRepmat`.

4.7 Indexing And Assignment

Parenthesis indexing selects a matrix block from every coefficient. Assignment writes the selected block into every coefficient after compatible promotion and degree alignment. It does not select a physical cell; use `coeffs` for cell inspection.

Syntax

Call forms

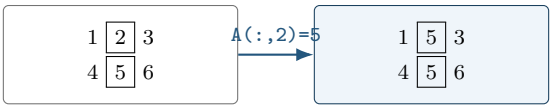
```
B = A(rows, cols)
B = A(1:end, end)
value = A.PropertyName
A(rows, cols) = numericValue
A(rows, cols) = B
```

MATLAB input

```
A = pdmat({[0 1]}, ...
    {[1 2 3; 4 5 6], [10 20 30; 40 50 60]}, Degree=1);
lastCol = A(:, end);
A(:, 2) = 5;
fprintf("selected size=%d-by-%d, assigned size=%d-by-%d\n", ...
    size(lastCol,1), size(lastCol,2), size(A,1), size(A,2));
disp(lastCol.evaluate(0))
disp(A.evaluate(0))
```

Output

```
selected size=2-by-1, assigned size=2-by-3
     3
     6

     1     5     3
     4     5     6
```



`subsref`, `subsasgn`, `end`, and `numArgumentsFromSubscript` implement this two-index MATLAB protocol. Invalid selectors raise `pdmat:InvalidSubscript`. Linear indexing, deletion, or non-parenthesis assignment raises `pdmat:UnsupportedAssignment`; an incompatible right-hand side raises `pdmat:InvalidAssignment`. Slicing and assignment require coefficient evidence and otherwise raise `pdmat:FunctionOnlyAlgebra`.

4.8 Limits

- Function-only objects created without `Degree` do not have Bernstein coefficient evidence for `bernsteinTable` or coefficient algebra.
- Numeric operands must be finite real nonempty matrices with compatible sizes, or finite real scalars that can broadcast to the required size.
- Grid refinement is supported for matching parameter dimensions and matching grid bounds.

4.8.1 See Also

`pdmat`, `evaluate`, `bernsteinTable`, `plot`, `pdbase`.

5

pdvar Reference

`pdvar` creates continuous cell-local Bernstein decision expressions backed by YALMIP `sdpvar` coefficients. It participates in supported affine and known-data algebra. It does not choose solvers or call `optimize`; solver-facing work happens after comparison overloads create `pdlmi` objects and `toYalmip` converts them to YALMIP constraints.

5.1 Lookup

Task	Entry
Construct decisions	`pdvar`
Inspect coefficients	`bernsteinTable`
Convert assigned coefficients	`value`
Differentiate rates	`rhodiff`
Affine/mixed algebra	`+`, `-`, `*` with known data
Matrix/index methods	structural methods, indexing and assignment
Build LMIs	`<=`, `>=`

5.2 Construction

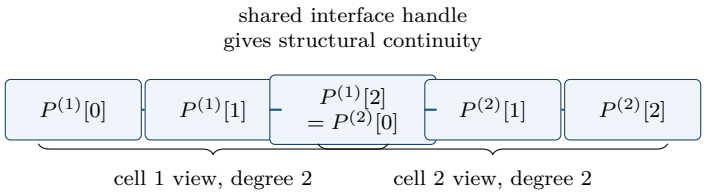
Syntax

Call forms

```
P = pdvar(n, gridVectors)
P = pdvar(n, n, gridVectors)
P = pdvar(n, m, gridVectors)
P = pdvar(..., "full")
P = pdvar(..., "symmetric")
P = pdvar(..., Degree=0)
P = pdvar(..., Degree=1)
P = pdvar(..., Degree=d)
P = pdvar(..., RateBounds=rb)
```

Arguments and options

- `pdvar(n, gridVectors)` creates an $n \times n$ square variable using scalar-size shorthand; for example, `pdvar(2, {[0 1]})`. Square variables default to symmetric YALMIP coefficients.
- `pdvar(n, n, gridVectors)` is the explicit square-size form; for example, `pdvar(2, 2, {[0 1]})`. It has the same default symmetric structure as the shorthand.
- `pdvar(n, m, gridVectors)` creates an $n \times m$ variable. Rectangular variables default to full YALMIP coefficients; for example, `pdvar(2, 3, {[0 1]})`.
- `"full"` and `"symmetric"` explicitly choose the YALMIP coefficient structure. `"symmetric"` requires a square size.
- `Degree=1` is the default continuous Bernstein decision variable. Every finite nonnegative integer `Degree=d` is supported. `Degree=0` creates one parameter-independent decision matrix shared across the grid; `d >= 1` creates continuous piecewise Bernstein data whose neighboring cells share complete control faces.
- `RateBounds=rb` stores parameter-rate metadata for later `rhodiff(P)` calls. Use `RateBounds=[-1 1]` for one parameter; use a two-row matrix for two parameters, as shown in the rate-metadata example below.



For positive degree, neighboring physical cells reuse complete boundary-face YALMIP handles; no extra continuity LMI is generated.

Example

The square shorthand creates a 2×2 continuous decision object. The default degree is one and the object carries no rate metadata.

MATLAB input

```
yalmip('clear')
Ps = pdvar(2, {[0 1]});
class(Ps)
size(Ps)
Ps.Degree
Ps.HasRateDependence
```

Output

```
ans =
    'pdvar'

ans =
     2     2

ans =
     1

ans =
  logical
   0
```

Use two dimension arguments for a rectangular full variable.

MATLAB input

```
yalmip('clear')
Pf = pdvar(2, 3, {[0 1]});
class(Pf)
size(Pf)
Pf.Degree
```

Output

```
ans =
    'pdvar'

ans =
     2     3

ans =
     1
```

The structure flag makes the coefficient structure explicit. This is useful when a square variable should use full, not symmetric, coefficients.

MATLAB input

```
yalmip('clear')
Pfull = pdvar(2, 2, {[0 1]}, "full");
class(Pfull)
Pfull.MatrixSize
Pfull.Degree
```

Output

```
ans =
    'pdvar'

ans =
     2     2

ans =
     1
```

Use `Degree=0` for a parameter-independent decision variable stored with grid metadata.

MATLAB input

```
yalmip('clear')
P0 = pdvar(1, [0 1 2], Degree=0);
class(P0)
P0.Degree
P0.ncell()
```

Output

```
ans =
    'pdvar'

ans =
     0

ans =
     2
```

Rate bounds are metadata for later `rhodiff(P)` calls. `pdlmi` constrains rate vertices only after an expression such as `rhodiff` has stored rate-vertex rows.

MATLAB input

```
yalmip('clear')
Pr = pdvar(1, {[0 1], [10 20]}, RateBounds=[-1 2; -3 4]);
class(Pr)
Pr.HasRateDependence
Pr.RateBounds
```

Output

```
ans =
    'pdvar'

ans =
  logical
   1

ans =
    -1     2
    -3     4
```

Validation and errors

Missing dimensions, a missing grid, or a grid in the wrong argument position raises `pdvar:InvalidInput`. Nonpositive or noninteger matrix sizes raise `pdvar:InvalidMatrixSize`; malformed degrees raise `pdvar:InvalidDegree`. Malformed option sequences raise `pdvar:InvalidOptions`; an unknown option raises `pdvar:UnknownOption`; attempts to set subclass-owned metadata raise `pdvar:UnsupportedOption`. A symmetric request for a nonsquare matrix raises `pdvar:InvalidStructure`. Grid validation uses `pdvar:InvalidGrid` and `pdvar:InvalidGridVector`.

`RateBounds` is validated by the inherited storage contract and must be a finite ℓ-by-2 matrix with lower bounds no greater than upper bounds. Constructor violations therefore raise `pdbase:InvalidRateBounds`.

5.3 `rhodiff`

Syntax

Call forms

```
D = rhodiff(P)
D = rhodiff(P, rb)
```

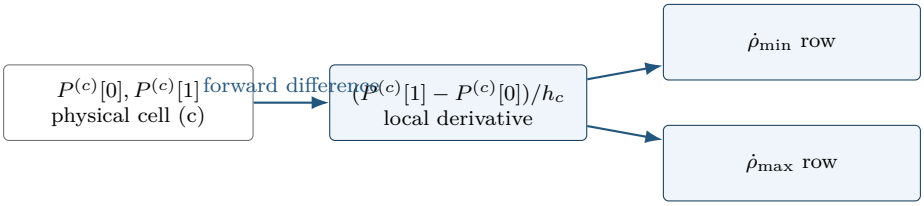
Arguments and options

- `P` is a `pdvar` without existing rate-vertex rows.
- `rb` is a finite ℓ-by-2 numeric matrix of lower and upper parameter-rate bounds, with each lower bound no greater than its upper bound. It must equal `P.RateBounds` when the object already carries nonempty bounds.
- Without `rb`, `rhodiff(P)` requires nonempty `P.RateBounds`.
- `D` is a discontinuous `pdvar` with one coefficient row per rate vertex. In one parameter a degree-m derivative has degree $m-1$, while a degree-zero derivative remains zero; multivariate partials are elevated into the implementation's common tensor degree.

For a one-parameter degree-one coefficient pair $P^{(c)}[0], P^{(c)}[1]$ on physical cell (c) of width h_c, the rate-dependent derivative row is

$$\dot{P}(\rho, \dot{\rho}) = \dot{\rho}\,\frac{P^{(c)}[1] - P^{(c)}[0]}{h_c}.$$

The implementation stores one row for each allowed rate vertex.



Differentiation changes local coefficients; rate substitution creates rows. Neither operation adds physical cells.

Validation and errors

Unsupported call arity or differentiating an expression that already contains rate-vertex rows raises `pdvar:InvalidDiff`. Explicit malformed rate bounds raise `pdvar:InvalidRateBounds`. Calling `rhodiff(P)` without stored bounds raises `pdvar:MissingRateBounds`; explicit bounds that disagree with `P.RateBounds` raise `pdvar:RateBoundsMismatch`.

Example

When rate bounds live on the object, `rhodiff(P)` uses them directly.

MATLAB input

```
yalmip('clear')
P = pdvar(2, {[0 1 2]}, "symmetric", RateBounds=[−1 1]);
D = rhodiff(P);
class(D)
D.Degree
D.IsContinuous
size(D.coeffs(1))
D.RateBounds
```

Output

```
ans =
    'pdvar'

ans =
     0

ans =
  logical
   0

ans =
     2     1

ans =
    -1     1
```

For a two-parameter object, the rate-vertex rows enumerate all lower/upper combinations from the two rows of `RateBounds`.

MATLAB input

```
yalmip('clear')
Q = pdvar(1, {[0 1], [10 20]}, RateBounds=[−1 1; −2 2]);
E = rhodiff(Q);
class(E)
E.Degree
size(E.coeffs([1 1]))
E.RateBounds
```

Output

```
ans =
    'pdvar'

ans =
     1

ans =
     4     4

ans =
```

```
    -1     1
    -2     2
```

5.4 bernsteinTable

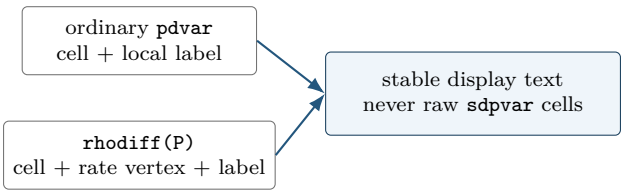
Syntax

Call forms

```
T = bernsteinTable(P)
T = bernsteinTable(P, cellSubs)
T = bernsteinTable(P, "oneLine")
T = bernsteinTable(P, cellSubs, "oneLine")
```

Arguments and options

- `P` is a `pdvar` object.
- `cellSubs` selects one physical cell; omit it to list every cell.
- `"oneLine"` is the only supported text option. It returns one compact Bernstein expression per selected cell.
- `T` is a table. Ordinary rows contain the local label, basis, physical-node flag, and symbolic or numeric value. A rate-dependent derivative also includes `RateVertexIndex` and `RateVertex`.



Example

MATLAB input

```
yalmip('clear')
P = pdvar(1, {[0 1]});
T = bernsteinTable(P, "oneLine");
disp(T)
```

Output

```
    CellSubscript       Expression
    -------------    ----------------

        {[1]}        "(1-alpha)*internal(1) + alpha*internal(2)"
```

Validation and errors

`bernsteinTable` is an inspection method, not a solver call. Text options other than `"oneLine"`, repeated selectors, invalid physical-cell selectors, or inconsistent rate-vertex rows raise `pdvar:InvalidBernsteinTableInput`.

5.5 Affine Algebra And Known-Data Products

A `pdvar` result must remain affine in the underlying YALMIP decisions. The following blocks separate affine sums from products so the supported boundary is visible beside each example.

5.5.1 Signs, Addition, And Subtraction

Use this family to combine compatible decision expressions, coefficient-backed known data, finite real numeric data, and affine `sdpvar` matrices. Operands with the same physical bounds are aligned on a common grid and degree. The corresponding overload names are `plus`, `minus`, `uplus`, and `uminus`.

Syntax

Call forms

```
R = P + Q
R = P + A
R = A + P
R = P + M
R = M + P
R = P + X
R = X + P
R = P - Q
R = P - A
R = A - P
R = P - M
R = M - P
R = +P
R = -P
```

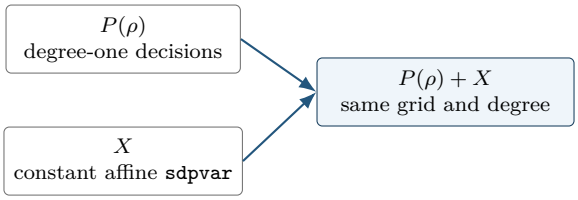
Here `A` is coefficient-backed `pdmat`, `M` is numeric, and `X` is affine `sdpvar`. A numeric scalar broadcasts to the payload size; an affine `sdpvar` is promoted as parameter-independent data.

MATLAB input

```
yalmip('clear')
P = pdvar(2, {[0 1]}, "full");
X = sdpvar(2, 2, 'full');
S = P + X;
D = eye(2) - P;
N = -P;
fprintf("sum: %s, %d-by-%d, degree=%d, decision=%d\n", ...
    class(S), size(S,1), size(S,2), S.Degree, S.ContainsDecision);
fprintf("difference: %s, unary minus: %s\n", class(D), class(N));
```

Output

```
sum: pdvar, 2-by-2, degree=1, decision=1
difference: pdvar, unary minus: pdvar
```



A nonlinear symbolic operand such as `x*x` is rejected. Rate-vertex rows from `rhodiff` may be added to compatible ordinary expressions; the ordinary rows are broadcast over the active rate vertices.

5.5.2 Multiplication By Known Or Numeric Data

Multiplication is supported only when decision dependence occurs on at most one side. A coefficient-backed `pdmat` or finite real numeric matrix may multiply a `pdvar` from either side. A scalar `pdvar` may scale a known matrix. Two decision-bearing factors, including `P*Q` and `P*x` for a decision `sdpvar x`, are nonlinear and rejected.

Syntax

Call forms

```
R = A * P
R = P * A
R = M * P
R = P * M
```

The first example shows numeric left and right matrix products without exposing incidental YALMIP variable identifiers.

MATLAB input

```
yalmip('clear')
P = pdvar(2, 1, {[0 1]}, "full");
L = [1 2] * P;
R = P * [4 5];
S = 3 * P;
fprintf("left=%d-by-%d, right=%d-by-%d, scaled=%d-by-%d\n", ...
    size(L,1), size(L,2), size(R,1), size(R,2), size(S,1), size(S,2));
fprintf("degrees: %d, %d, %d\n", L.Degree, R.Degree, S.Degree);
```

Output

```
left=1-by-1, right=2-by-2, scaled=2-by-1
degrees: 1, 1, 1
```

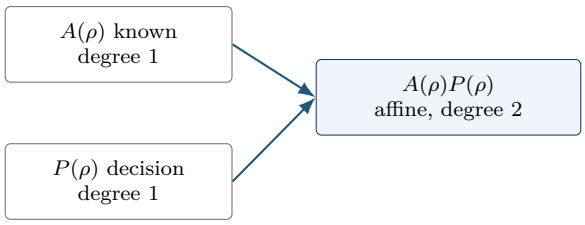
A known parameter-dependent factor adds its Bernstein degree through ordered coefficient convolution.

MATLAB input

```
yalmip('clear')
P = pdvar(2, {[0 1]}, "symmetric");
A = pdmat({[0 1]}, {eye(2), 2*eye(2)}, Degree=1);
L = A * P;
R = P * A;
fprintf("known-left: %s degree %d; known-right: %s degree %d\n", ...
    class(L), L.Degree, class(R), R.Degree);
fprintf("coefficients per cell=%d\n", numel(L.coeffs(1)));
```

Output

```
known-left: pdvar degree 2; known-right: pdvar degree 2
coefficients per cell=3
```



A derivative object may be multiplied by numeric data or a matching-grid known object while preserving every rate row. Products with rate dependence on both sides, products of a derivative with an ordinary decision expression, and products involving metadata-only rate dependence are rejected.

Validation and errors

Incompatible affine operands raise `pdvar:InvalidAddition` or `pdvar:InvalidSubtraction`. Bad dimensions, two decision-bearing factors, nonlinear symbolic data, or unsupported rate combinations raise `pdvar:InvalidMultiplication`. Incompatible parameter bounds raise `pdvar:MixedGrid`; a function-only `pdmat` operand raises `pdvar:FunctionOnlyAlgebra`.

5.6 Matrix Methods

Every method below transforms each local affine YALMIP payload while preserving the physical grid, degree, and compatible rate rows. The examples report stable classes, dimensions, and metadata rather than YALMIP's internal variable numbers.

5.6.1 Shape, Equality, And MATLAB Protocols

Shape queries describe the matrix payload. `isequal` is an exact identity check: grid, degree, matrix size, metadata, and local symbolic payloads must already match; it does not merge or elevate operands.

Syntax

Call forms

```
sz = size(P)
d = size(P, dim)
n = height(P)
n = width(P)
n = length(P)
n = numel(P)
n = ndims(P)
tf = isequal(P, Q, ...)
idx = end(P, k, n)
n = numArgumentsFromSubscript(P, S, ctx)
```

MATLAB input

```
yalmip('clear')
P = pdvar(2, 3, {[0 1]}, "full");
fprintf("size=%d-by-%d, height=%d, width=%d\n", ...
    size(P,1), size(P,2), height(P), width(P));
fprintf("length=%d, numel=%d, ndims=%d, equal(+P)=%d\n", ...
    length(P), numel(P), ndims(P), isequal(P,+P));
```

Output

```
size=2-by-3, height=2, width=3
length=3, numel=6, ndims=2, equal(+P)=1
```

`end`, `subsref`, `subsasgn`, and `numArgumentsFromSubscript` support the two-index and dot-access forms shown in section 5.7; they do not expose physical-cell storage.

5.6.2 Transpose And Conjugate Transpose

Both forms transpose every local affine payload without breaking the YALMIP coefficient graph.

Syntax

Call forms

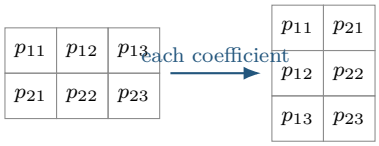
```
T = transpose(P)
H = ctranspose(P)
T = P.'
H = P'
```

MATLAB input

```
yalmip('clear')
P = pdvar(2, 3, {[0 1]}, "full");
T = P.';
H = P';
fprintf("transpose=%d-by-%d, ctranspose=%d-by-%d\n", ...
    size(T,1), size(T,2), size(H,1), size(H,2));
fprintf("classes: %s, %s\n", class(T), class(H));
```

Output

```
transpose=3-by-2, ctranspose=3-by-2
classes: pdvar, pdvar
```



5.6.3 Vectorization, Reshape, And Squeeze

`vec` uses MATLAB column-major order. `reshape` accepts two positive dimensions with at most one inferred `[]` dimension and preserves the element count. `squeeze` retains the two-dimensional package convention.

Syntax

Call forms

```
V = vec(P)
R = reshape(P, [m n])
R = reshape(P, m, n)
S = squeeze(P)
```

MATLAB input

```
yalmip('clear')
P = pdvar(2, 3, {[0 1]}, "full");
V = vec(P);
R = reshape(P, [3 2]);
S = squeeze(P);
fprintf("vec=%d-by-%d, reshape=%d-by-%d, squeeze=%d-by-%d\n", ...
    size(V,1), size(V,2), size(R,1), size(R,2), size(S,1), size(S,2));
```

Output

```
vec=6-by-1, reshape=3-by-2, squeeze=2-by-3
```

Malformed or size-changing reshape requests raise `pdvar:InvalidReshape`.

5.6.4 Diagonals And Trace

`diag` extracts a diagonal or constructs a diagonal matrix from a vector; the selected offset may not be empty. `trace` returns a scalar decision expression.

Syntax

Call forms

```
d = diag(P)
d = diag(P, k)
D = diag(v)
D = diag(v, k)
t = trace(P)
```

MATLAB input

```
yalmip('clear')
P = pdvar(3, {[0 1]}, "full");
d = diag(P);
D = diag(d);
t = trace(P);
fprintf("diag=%d-by-%d, rebuilt=%d-by-%d, trace=%d-by-%d\n", ...
    size(d,1), size(d,2), size(D,1), size(D,2), size(t,1), size(t,2));
```

Output

```
diag=3-by-1, rebuilt=3-by-3, trace=1-by-1
```

Invalid offsets or empty selections raise `pdvar:InvalidDiag`.

5.6.5 Triangular Parts And Reductions

Triangular methods zero the complementary part of every coefficient. Reductions act on matrix dimensions; `"all"` is available for `sum` and `mean`.

Syntax

Call forms

```
L = tril(P)
L = tril(P, k)
U = triu(P)
U = triu(P, k)
S = sum(P)
S = sum(P, dim)
S = sum(P, "all")
M = mean(P)
M = mean(P, dim)
M = mean(P, "all")
C = cumsum(P)
C = cumsum(P, dim)
```

First form the two triangular views.

MATLAB input

```
yalmip('clear')
P = pdvar(3, {[0 1]}, "full");
L = tril(P);
U = triu(P);
fprintf("lower=%d-by-%d, upper=%d-by-%d, degrees=%d/%d\n", ...
    size(L,1), size(L,2), size(U,1), size(U,2), L.Degree, U.Degree);
```

Output

```
lower=3-by-3, upper=3-by-3, degrees=1/1
```

Then reduce a rectangular expression.

MATLAB input

```
yalmip('clear')
P = pdvar(2, 3, {[0 1]}, "full");
rowSum = sum(P, 2);
allMean = mean(P, "all");
running = cumsum(P, 2);
fprintf("row sum=%d-by-%d, all mean=%d-by-%d, running=%d-by-%d\n", ...
    size(rowSum,1), size(rowSum,2), size(allMean,1), size(allMean,2), ...
    size(running,1), size(running,2));
```

Output

```
row sum=2-by-1, all mean=1-by-1, running=2-by-3
```

Invalid triangular offsets raise `pdvar:InvalidTriangularPart`. Invalid reduction calls raise `pdvar:InvalidSum`, `pdvar:InvalidMean`, or `pdvar:InvalidCumsum`.

5.6.6 Reordering And Rotation

These methods rearrange matrix entries inside every symbolic coefficient; they do not reverse the parameter grid.

Syntax

Call forms

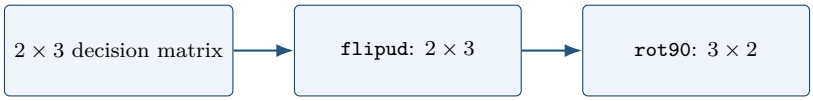
```
Q = flip(P)
Q = flip(P, dim)
Q = flipud(P)
Q = fliplr(P)
Q = rot90(P)
Q = rot90(P, k)
```

MATLAB input

```
yalmip('clear')
P = pdvar(2, 3, {[0 1]}, "full");
F = flipud(P);
R = rot90(P);
fprintf("flipud=%d-by-%d, rot90=%d-by-%d\n", ...
    size(F,1), size(F,2), size(R,1), size(R,2));
fprintf("classes: %s, %s\n", class(F), class(R));
```

Output

```
flipud=2-by-3, rot90=3-by-2
classes: pdvar, pdvar
```



Invalid calls raise `pdvar:InvalidFlip` or `pdvar:InvalidRot90`.

5.6.7 Concatenation And Block Assembly

Assembly accepts compatible `pdvar`, coefficient-backed `pdmat`, affine `sdpvar`, and finite real numeric blocks. It aligns grids and degrees while preserving an affine expression. `cat` supports only matrix dimensions 1 and 2.

Syntax

Call forms

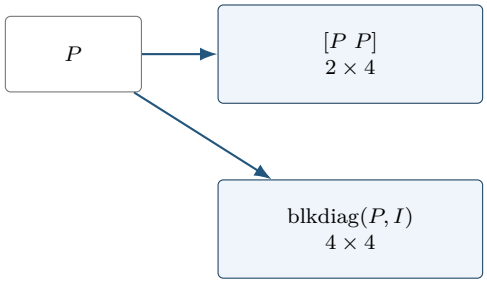
```
H = horzcat(P, Q, ...)
V = vertcat(P, Q, ...)
H = [P, Q]
V = [P; Q]
C = cat(dim, P, Q, ...)
D = blkdiag(P, Q, ...)
```

MATLAB input

```
yalmip('clear')
P = pdvar(2, {[0 1]}, "full");
H = [P, P];
V = [P; P];
D = blkdiag(P, eye(2));
fprintf("horizontal=%d-by-%d, vertical=%d-by-%d, block=%d-by-%d\n", ...
    size(H,1), size(H,2), size(V,1), size(V,2), size(D,1), size(D,2));
```

Output

```
horizontal=2-by-4, vertical=4-by-2, block=4-by-4
```



Incompatible blocks or rate metadata raise `pdvar:InvalidConcatenation`; unsupported dimensions raise
`pdvar:UnsupportedCatDimension`; invalid block-diagonal inputs raise `pdvar:InvalidBlkdiag`. Incompatible bounds raise `pdvar:MixedGrid`; function-only known blocks raise `pdvar:FunctionOnlyAlgebra`.

5.6.8 Repetition

`repmat` tiles the matrix payload while retaining the parameter metadata. It accepts a positive scalar, two positive counts, or a two-element count vector.

Syntax

Call forms

```
Q = repmat(P, m)
Q = repmat(P, m, n)
Q = repmat(P, [m n])
```

MATLAB input

```
yalmip('clear')
P = pdvar(1, 2, {[0 1]}, "full");
Q = repmat(P, [2 2]);
fprintf("repeated=%d-by-%d, degree=%d, decision=%d\n", ...
    size(Q,1), size(Q,2), Q.Degree, Q.ContainsDecision);
```

Output

```
repeated=2-by-4, degree=1, decision=1
```

Invalid repetition shapes raise `pdvar:InvalidRepmat`.

5.7 Indexing And Assignment

Parenthesis indexing selects a matrix block from every symbolic coefficient. Assignment writes a compatible numeric, `pdvar`, coefficient-backed `pdmat`, or affine `sdpvar` block into that position. It does not select one physical cell.

Syntax

Call forms

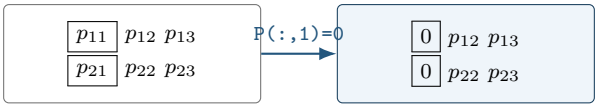
```
Q = P(rows, cols)
Q = P(1:end, end)
value = P.PropertyName
P(rows, cols) = numericValue
P(rows, cols) = Q
P(rows, cols) = A
P(rows, cols) = X
```

MATLAB input

```
yalmip('clear')
P = pdvar(2, 3, {[0 1]}, "full");
lastCol = P(:, end);
P(:, 1) = 0;
fprintf("selected=%d-by-%d, assigned=%d-by-%d\n", ...
    size(lastCol,1), size(lastCol,2), size(P,1), size(P,2));
fprintf("class=%s, degree=%d, decision=%d\n", ...
    class(P), P.Degree, P.ContainsDecision);
```

Output

```
selected=2-by-1, assigned=2-by-3
class=pdvar, degree=1, decision=1
```



Invalid selectors raise `pdvar:InvalidSubscript`. Linear indexing, deletion, or non-parenthesis assignment raises `pdvar:UnsupportedAssignment`; incompatible or nonlinear right-hand sides raise `pdvar:InvalidAssignment`.

5.8 `value`

Syntax

Call forms

```
A = value(P)
rows = value(rhodiff(P))
```

`value` converts assigned `pdvar` coefficients into known, coefficient-backed `pdmat` data. An ordinary expression returns one `pdmat`, preserving its grid, matrix size, degree, local coefficient order, and continuity metadata. A rate-vertex expression created by `rhodiff` returns a 1-by-2^{ℓ} cell array of `pdmat` objects in `helper.combRows(RateBounds)` lower/upper vertex order. The returned `pdmat` objects do not carry rate metadata. Every symbolic coefficient must have an assigned finite real YALMIP value; otherwise the method raises `pdvar:UnassignedValue`.

Example

MATLAB input

```
yalmip('clear')
P = pdvar(1, [0 1], Degree=2);
c = P.coeffs(1);
assign([c{:}], [1 2 4]);
A = value(P);
A.coeffs(1)
```

Output

```
ans =
    {[1]}    {[2]}    {[4]}
```

5.9 Comparisons To `pdlmi`

The `le` and `ge` overloads implement `<=` and `>=`.

Syntax

Call forms

```
C = P <= 0
C = P >= 0
C = lhs <= rhs
C = lhs >= rhs
```

Arguments and options

- `P` and `lhs` are square, coefficient-wise symmetric or Hermitian `pdvar` expressions after residual assembly.
- `rhs` is commonly `0`; compatible numeric, `pdmat`, affine `sdpvar`, and `pdvar` expressions are also accepted.
- `C` is a `pdlmi` wrapper. The comparison creates direct coefficient constraints but does not solve an optimization problem.

Example

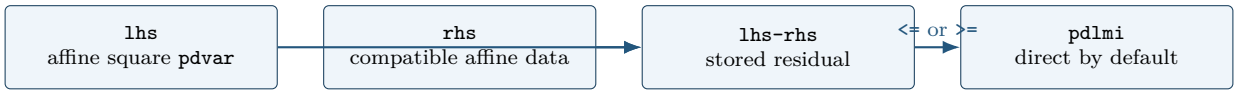
Comparison creates a `pdlmi` wrapper with one stored constraint for each physical cell and local Bernstein coefficient.

MATLAB input

```
yalmip('clear')
P = pdvar(2, {[0 0.5 1]}, "symmetric");
Cneg = P <= 0;
class(Cneg)
numel(Cneg.Constraints)
Cpos = P >= 0;
class(Cpos)
numel(Cpos.Constraints)
```

Output

```
ans =
    'pdlmi'

ans =
     4

ans =
    'pdlmi'

ans =
     4
```



5.10 Limits

- Constructor-created `pdvar` objects support every finite nonnegative integer degree. Higher degree increases the number of Bernstein control matrices per cell.

- Products may contain decision variables on at most one side; nonlinear decision products such as `P * Q` are outside this slice.
- Function-only `pdmat` operands cannot enter `pdvar` algebra unless they were constructed with explicit Bernstein evidence.
- `rhodiff` of an existing rate-vertex derivative expression is not part of the current public workflow.

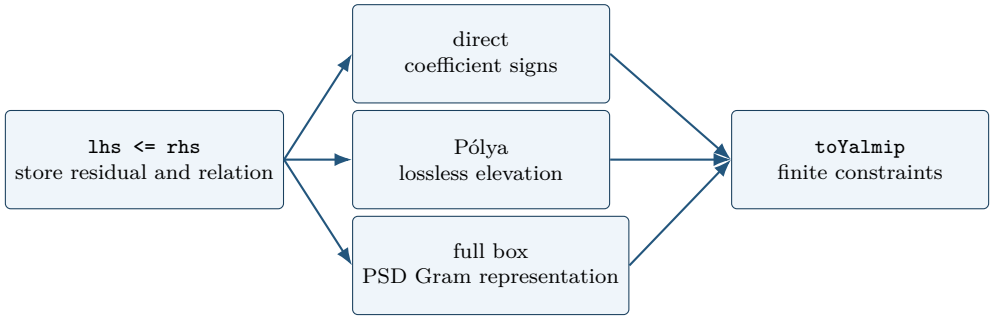
5.10.1 See Also

`pdvar`, `rhodiff`, `bernsteinTable`, `pdlmi`, `pdmat`.

6

pdlmi Reference

pdlmi stores YALMIP constraints assembled from square symmetric pdvar expressions. Direct coefficient-wise assembly is the default; cell-local Pólya elevation and the full box Bernstein–Gram preordering are explicit alternatives. It is the package boundary between Bernstein objects and ordinary YALMIP solver calls.



The three certificate paths are alternatives. Selecting Pólya or full-box rebuilding replaces, rather than compounds, the previous selection.

6.1 Lookup

Task	Entry
Construct wrapper	pdlmi, <=/>=
Count constraints	direct coefficient constraints
Select Pólya elevation	UsePolya, PolyaDegree, and applyPolya
Select full box preordering	UseFullBoxPreorder, FullBoxOrder, and applyFullBoxPreorder
Export to YALMIP	toYalmip
Solve	solver-facing use, Examples

6.2 Construction And Comparisons

Syntax

Call forms

```
C = pdlmi(expr, relation)
C = pdlmi(expr, relation, "UsePolya")
C = pdlmi(expr, relation, UsePolya=true, PolyaDegree=d)
C = pdlmi(expr, relation, "UsePolya", "PolyaDegree", d)
C = pdlmi(expr, relation, "UseFullBoxPreorder")
```

```
C = pdlmi(expr, relation, UseFullBoxPreorder=true)
C = pdlmi(expr, relation, FullBoxOrder=r)
C = pdlmi(expr, relation, UseFullBoxPreorder=true, FullBoxOrder=r)
C = pdlmi(expr, relation, UseFullBoxPreorder=false, FullBoxOrder=r)
C = lhs <= rhs
C = lhs >= rhs
```

Arguments and options

- `expr` is a square `pdvar` expression whose coefficient matrices are symmetric or Hermitian; `relation` is exactly `"<="` or `">="`.
- `UsePolya` is a logical scalar option with default `false`. Its bare form, `"UsePolya"`, and `UsePolya=true` without a degree select increment one.
- `PolyaDegree=d` is a finite nonnegative integer scalar. It elevates the residual by `d` in every parameter direction before assembling constraints. Supplying it without `UsePolya` enables Pólya and issues `pdlmi:ImplicitUsePolya`; `UsePolya=false` with positive `d` raises `pdlmi:ConflictingPolyaOptions`.
- `UseFullBoxPreorder` is a logical scalar option with default `false`. Its bare form and the explicit value `true` select full-box assembly at the smallest admissible order.
- `FullBoxOrder=r` is a finite nonnegative integer scalar and is an absolute order, not a degree increment. Supplying it without `UseFullBoxPreorder` enables full-box assembly. When full-box assembly is enabled, the minimum is $\lfloor m/2 \rfloor$ for a one-parameter residual of degree m, and $\lceil m/2 \rceil$ for two or more parameters. Explicitly pairing `UseFullBoxPreorder=false` with `FullBoxOrder=r` suppresses that automatic selection: `C` remains direct and records `r` only as inspectable metadata; the order is not applied to assembly.
- Pólya and full-box selection are mutually exclusive. Enabling both raises `pdlmi:ConflictingRelaxations`.
- `C` is direct coefficient-wise assembly unless one opt-in mode is selected. Comparisons such as `lhs >= rhs` are the ordinary user-facing entry point and accept compatible numeric, `pdmat`, affine `sdpvar`, and `pdvar` right-hand sides.

The inspectable properties `Constraints`, `Residual`, and `Relation` expose the assembled finite constraints and the original comparison data used for rebuilding. `UsePolya`, `PolyaDegree`, `UseFullBoxPreorder`, and `FullBoxOrder` report the selected mode and order metadata. Their set access is private. Comparison-created direct objects, and direct constructor calls that omit `FullBoxOrder`, have both selectors false and both numeric properties zero. The explicit-false combination described above is the exception: it retains `FullBoxOrder=r` while `UseFullBoxPreorder` remains false and the stored constraints remain direct.

Example

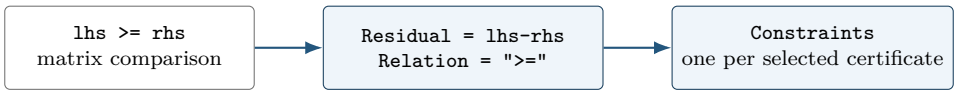
The direct constructor is equivalent to the comparison path when the relation and options are explicit.

MATLAB input

```
yalmip('clear')
P = pdvar(2, {[0 1]}, "symmetric");
C = pdlmi(P, ">=", UsePolya=false, PolyaDegree=0);
class(C)
numel(C.Constraints)
C.UsePolya
C.PolyaDegree
C.UseFullBoxPreorder
C.FullBoxOrder
```

Output

```
ans =
    'pdlmi'

ans =
     2

ans =
  logical
   0

ans =
     0

ans =
  logical
   0

ans =
     0
```



Validation and errors

The constructor rejects a non-`pdvar` residual with `pdlmi:InvalidExpression`, a relation other than `"<="` or `">="` with `pdlmi:InvalidRelation`, and malformed, unknown, or duplicate options with `pdlmi:InvalidOptions`, `pdlmi:UnknownOption`, or `pdlmi:DuplicateOption`. Nonlogical selectors raise `pdlmi:InvalidUsePolya` or `pdlmi:InvalidUseFullBoxPreorder`. Malformed Pólya increments and full-box orders raise `pdlmi:InvalidPolyaDegree` and `pdlmi:InvalidFullBoxOrder`, respectively. When full-box assembly is enabled, a valid integer order below the dimension-dependent minimum instead raises `pdlmi:FullBoxOrderTooLow`. An explicitly false selector keeps assembly direct, so this admissibility minimum is not applied to its retained order metadata. Every assembly

path retains the square and symmetry checks, which raise `pdlmi:InvalidMatrixSize` and `pdlmi:NonSymmetricExpression`.

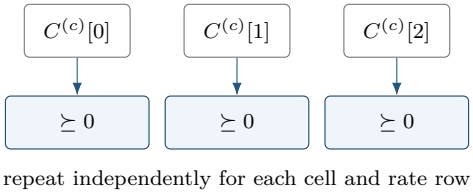
6.3 Direct Coefficient Assembly

For each physical cell, local Bernstein coefficient, and rate-vertex row if present, `pdlmi` creates one YALMIP semidefinite constraint. For an ordinary expression with N_c physical cells and N_b Bernstein coefficients per cell, the number of stored constraints is $N_c N_b$. For a `rhodiff` expression with N_v rate vertices, the count becomes $N_c N_b N_v$.

The idea is the Bernstein convex-hull property. After orienting the relation as a positive-semidefinite target

$$S_{\boldsymbol{c},v}(\boldsymbol{\alpha}) = \sum_{\boldsymbol{i}} B^m_{\boldsymbol{i}}(\boldsymbol{\alpha}) C^{(\boldsymbol{c},v)}[\boldsymbol{i}],$$

the direct certificate requires $C^{(\boldsymbol{c},v)}[\boldsymbol{i}] \succeq 0$ for every physical cell $\boldsymbol{c}$, active rate vertex v, and local label $\boldsymbol{i}$. This is sufficient, not necessary.



6.4 Opt-In Pólya Assembly And `applyPolya`

With `UsePolya=true`, `pdlmi` first elevates the stored residual by `PolyaDegree` in every parameter direction, then constrains every elevated local coefficient and every active rate row. Thus, if the original degree is m, the count is $N_c(m+d+1)^\ell$ for ordinary expressions and $N_c(m+d+1)^\ell N_v$ for rate-vertex expressions. An increment of zero is valid and has the same coefficient count as direct assembly.

Syntax

Call forms

```
Cpolya = C.applyPolya()
Cpolya = C.applyPolya(d)
```

`applyPolya` is a value-class method: it returns a new `pdlmi`, leaves `C` unchanged, and rebuilds from `C.Residual`. Selecting a new increment therefore replaces rather than compounds a previous selection. The no-argument form uses one; an invalid increment raises `pdlmi:InvalidPolyaDegree`.

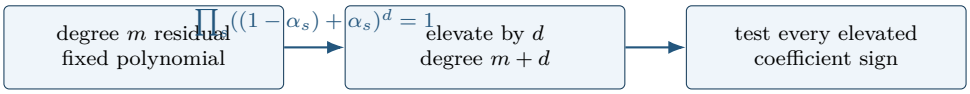
Example

MATLAB input

```
yalmip('clear')
P = pdvar(1, {[0 1]}, Degree=1);
direct = P >= 0;
polya = direct.applyPolya(2);
direct.PolyaDegree
polya.UsePolya
polya.PolyaDegree
numel(polya.Constraints)
```

Output

```
ans =
     0

ans =
  logical
   1

ans =
     2

ans =
     4
```



Pólya degree is a uniform representation increment, not a new decision degree, a grid refinement, or a Gram order.

The mathematical background and the limits of fixed-increment coefficient positivity are developed in section B.

6.5 Full Box Bernstein–Gram Preordering And `applyFullBoxPreorder`

The full-box mode is a cell-local positive-semidefinite representation in the forward local coordinates $\alpha_s \in [0, 1]$. It is a box-specific Bernstein–Gram preordering, not a general Putinar certificate, general-domain SOS interface, or full-space SOS representation. It adds no implicit positivity margin.

Syntax

Call forms

```
Cbox = C.applyFullBoxPreorder()
Cbox = C.applyFullBoxPreorder(r)
```

The no-argument form selects the smallest admissible absolute order. The explicit form selects `r`; it does not add `r` to the residual degree. Both forms are value-class selectors: they return a new `pdlmi`, leave `C` unchanged, and rebuild from `C.Residual`. Reapplication therefore replaces an earlier full-box order or a Pólya selection rather than compounding it.

The idea is to represent the oriented positive target as a sum of PSD Gram forms multiplied by functions that are nonnegative on the local box:

$$S(\boldsymbol{\alpha}) = \sum_{J \subseteq \{1,\dots,\ell\}} \left(\prod_{s \in J} \alpha_s (1 - \alpha_s) \right) \mathcal{Z}_J(\boldsymbol{\alpha})^{\mathsf{T}} Q_J \mathcal{Z}_J(\boldsymbol{\alpha}), \qquad Q_J \succeq 0.$$

In one parameter the implementation uses the even/odd Markov–Lukács forms, with minimum absolute order $\lfloor m/2 \rfloor$ for residual degree m. For two or more parameters it uses every subset product of the ℓ box generators and minimum order $\lceil m/2 \rceil$. The selected order fixes the dense Bernstein Gram bases; it is not added to m.

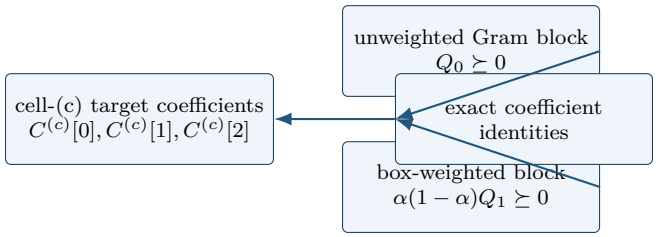
MATLAB input

```
yalmip('clear')
P = pdvar(1, {[0 1]}, Degree=2);
direct = P >= 0;
box = direct.applyFullBoxPreorder();
fprintf("direct constraints=%d\n", numel(direct.Constraints));
fprintf("full-box=%d, order=%d, constraints=%d\n", ...
    box.UseFullBoxPreorder, box.FullBoxOrder, numel(box.Constraints));
```

Output

```
direct constraints=3
full-box=1, order=1, constraints=5
```

At this order the scalar degree-two cell has two PSD Gram constraints followed by three exact Bernstein coefficient identities.



Appendix B derives the one-dimensional parity forms, explains the multivariate subset masks, and distinguishes this implemented box preordering from full-space SOS, Putinar modules, and general

polynomial parsers. Assembly is independent for every physical cell and every active rate-vertex row; Gram variables are not shared across either boundary. For `>=`, the method represents the residual. For `<=`, it negates the target before forming the positive-semidefinite representation. Within each cell and row, the stored constraints are ordered as all PSD Gram blocks followed by exact equalities matching every Bernstein coefficient. Consequently, the number of stored constraints is not the direct coefficient count.

Validation and errors

Malformed orders raise `pdlmi:InvalidFullBoxOrder`; an admissible integer below the minimum raises `pdlmi:FullBoxOrderTooLow`. The constructor rejects malformed selectors with `pdlmi:InvalidUseFullBoxPreorder`, and simultaneous Pólya and full-box selection raises `pdlmi:ConflictingRelaxations`. The usual expression, relation, option, square-matrix, and symmetry checks described in section 6.2 apply unchanged.

6.6 `toYalmip`

Syntax

Call forms

```
F = toYalmip(C)
F = C.toYalmip()
```

Arguments and options

`toYalmip` concatenates the stored constraint cells into a YALMIP constraint array for direct, Pólya, and full-box objects. It has no package-specific options and does not add a solver wrapper, objective, margin, or diagnostic layer. The output is suitable for `optimize`, concatenation with other YALMIP constraints, or ordinary YALMIP inspection.

Example

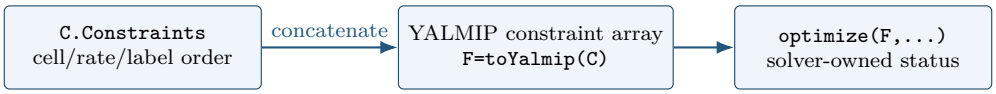
The function-call and dot-call forms return equivalent YALMIP constraint arrays.

MATLAB input

```
yalmip('clear')
P = pdvar(2, {[0 1]}, "symmetric");
C = P >= 0;
class(C)
numel(C.Constraints)
F1 = toYalmip(C);
F2 = C.toYalmip();
isa(F1, "lmi") || isa(F1, "constraint")
length(F1)
isa(F2, "lmi") || isa(F2, "constraint")
length(F2)
```

Output

```
ans =
    'pdlmi'

ans =
     2

ans =
  logical
   1

ans =
     2

ans =
  logical
   1

ans =
     2
```



After this boundary, the package is no longer assembling Bernstein data; YALMIP owns solver choice, diagnostics, and optimization status.

6.7 See Also

`pdvar`, `rhodiff`, `applyPolya`, `applyFullBoxPreorder`, `toYalmip`, `optimize`.

6.8 Solver-Facing Use

`pdlmi` does not own solver settings or wrap `optimize`. After `toYalmip`, use ordinary YALMIP:

Call forms

```
opts = sdpsettings("solver", solverName, "verbose", 0);
sol = optimize([C1.toYalmip, C2.toYalmip], objective, opts);
```

The solver selected by YALMIP must be available on the MATLAB path. The current test suite prefers MOSEK when available and otherwise falls back to LMILAB for smoke coverage.

7

Cross-Class Helper Reference

The `+helper` namespace contains exactly nine implementation utilities whose ownership genuinely crosses class boundaries. They are developer-facing building blocks, not an alternative modeling API: ordinary users should enter through `pdbase`, `pdmat`, `pdvar`, and `pdlmi`. Their signatures and current consumers are listed below so maintainers can distinguish shared contracts from class-local implementation details.

1. `helper.bernTbl(...)`
   Builds ordinary, rate-vertex, or one-line Bernstein inspection tables with caller-supplied value and expression formatters. *Consumers:* `pdmat`, `pdvar`.

2. `helper.cellGet(vals, subs)`
   Follows one tensor-subscript row through nested `LocalValues` and returns the selected leaf. *Consumers:* all four classes.

3. `helper.chk(val, id, msg, ...)`
   Applies shared predicate tags and bounds, returning the input or raising the caller's error ID. *Consumers:* all four classes.

4. `helper.combRows(vecs)`
   Returns Cartesian-product rows with earlier dimensions varying more slowly. *Consumers:* all four classes.

5. `helper.isZero(val, mode, ...)`
   Classifies numeric, additive, nested-payload, or coefficient-backed object zero evidence. *Consumers:* `pdmat`, `pdvar`.

6. `helper.mapVals(vals, fcn, grid)`
   Maps one function over every coefficient in a nested cell tree. *Consumers:* `pdmat`, `pdvar`.

7. `helper.matSubs(subs, sz, id)`
   Normalizes numeric, logical, or colon matrix subscripts into row and column indices. *Consumers:* `pdmat`, `pdvar`.

8. `helper.mkGrid(grid, owner)`
   Validates tensor-grid vectors and builds `GridInfo` with vectors, points, bounds, and node counts. *Consumers:* `pdbase`, `pdmat`, `pdvar`.

9. `helper.mkNest(counts, fcn[, prefix])`
   Constructs recursive tensor storage and calls the leaf factory with each complete subscript row. *Consumers:* `pdbase`, `pdmat`, `pdvar`.

7.1 Shared Validation And Errors

`helper.chk` recognizes the predicate tags `numeric`, `real`, `cell`, `struct`, `nonempty`, `scalar`, `vector`, `matrix`, `finite`, `integer`, `positive`, `nonnegative`, `increasing`, and `rowbounds`; its named tests are `Size`, `Numel`, `MinNumel`, `Min`, and `Max`. Failed user-data checks raise the supplied class error ID. An unknown tag or a missing validator-option value raises `helper:InvalidValidatorCall`.

`helper.isZero` accepts modes `"num"`, `"add"`, `"vals"`, and `"obj"`. Unknown modes raise `helper:InvalidZeroMode`. A wrong number of mode-specific inputs raises `helper:InvalidZeroCall`. `helper.matSubs` and `helper.bernTbl` deliberately receive a caller-owned error ID, so public indexing and table errors remain in the owning class namespace.

7.2 Class-Private Helpers

Single-class mechanics remain beside their owner rather than in `+helper`. The private area of `pdbase` owns coefficient-tree validation. The private area of `pdmat` owns source fitting, continuity, grid conversion, matrix scanning, and algebraic wrapping. The private area of `pdvar` owns affine and rate-row packing, product routing, value-tree zipping, and symbolic wrapping. The private area of `pdlmi` owns direct and full-box assembly, Gram conversion, and order validation. These functions have no supported direct call form. Their placement is an ownership boundary: a helper should move into a class when only that class can meaningfully use it, and should enter `+helper` only after a real cross-class contract exists.

8

Examples

8.1 Deterministic Full-Box Selection

This example selects the default and an explicit full-box order without solving an optimization problem. The reported values depend only on the public assembly contract, not on incidental YALMIP decision-variable identifiers.

MATLAB input

```
yalmip('clear')
P = pdvar(1, {[0 1]}, Degree=2);
direct = P >= 0;
boxDefault = direct.applyFullBoxPreorder();
boxHigher = boxDefault.applyFullBoxPreorder(2);
polya = direct.applyPolya(1);
boxFromPolya = polya.applyFullBoxPreorder();
Fbox = toYalmip(boxDefault);

fprintf("direct: fullBox=%d, order=%d, constraints=%d\n", ...
    direct.UseFullBoxPreorder, direct.FullBoxOrder, ...
    numel(direct.Constraints));
fprintf("default: fullBox=%d, order=%d, constraints=%d\n", ...
    boxDefault.UseFullBoxPreorder, boxDefault.FullBoxOrder, ...
    numel(boxDefault.Constraints));
fprintf("higher: order=%d, constraints=%d\n", ...
    boxHigher.FullBoxOrder, numel(boxHigher.Constraints));
fprintf("replacement: polya=%d, fullBox=%d, order=%d\n", ...
    boxFromPolya.UsePolya, boxFromPolya.UseFullBoxPreorder, ...
    boxFromPolya.FullBoxOrder);
fprintf("exported constraints=%d\n", length(Fbox));
```

Output

```
direct: fullBox=0, order=0, constraints=3
default: fullBox=1, order=1, constraints=5
higher: order=2, constraints=7
replacement: polya=0, fullBox=1, order=1
exported constraints=5
```

For the degree-two scalar residual, order one uses two PSD Gram blocks followed by three exact coefficient identities. Order two matches a degree-four target and stores two PSD blocks followed by five identities. The original `direct` object remains unchanged, and the selection made from `polya` replaces Pólya rather than elevating its already assembled constraints.

8.2 Parameter-Dependent Lyapunov Solver Smoke

The first solver smoke test in `+tests/+pdlmi/test_solver_smoke.m` contrasts a constant Lyapunov matrix with a degree-one, parameter-dependent one. It solves the same decay and positivity constraints in both cases. The degree-one solution is then materialized as known `pdmat` data and plotted. `pdvar` is a decision-expression class and does not itself provide `plot`; applying `value` to its local YALMIP payloads after a successful solve supplies the numeric Bernstein coefficients for `pdmat`.

MATLAB input

```
yalmip('clear')
grid = {[0 1]};
A = pdmat(grid, @(x) (1 - x) * [-1 -1; 1 -1] + ...
    x * [-1 -10; 0.1 -1], Degree=1);
solver = 'lmilab';
if exist('mosekopt', 'file') ~= 0
    solver = 'mosek';
end
opts = sdpsettings('solver', solver, 'verbose', 0);

Pc = pdvar(2, grid, Degree=0);
CconstantDecay = Pc * A + A' * Pc <= -1e-10 * eye(2);
CconstantPositive = Pc >= eye(2);
solConstant = optimize([CconstantDecay.toYalmip, ...
    CconstantPositive.toYalmip], [], opts);

Pd = pdvar(2, grid);
CdependentDecay = Pd * A + A' * Pd <= -1e-10 * eye(2);
CdependentPositive = Pd >= eye(2);
solDependent = optimize([CdependentDecay.toYalmip, ...
    CdependentPositive.toYalmip], [], opts);
assert(solDependent.problem == 0, solDependent.info)

Pplot = pdmat(grid, ...
    {cellfun(@value, Pd.LocalValues{1}, UniformOutput=false)}, ...
    Degree=Pd.Degree);
figure(Name="Parameter-dependent Lyapunov matrix")
plot(Pplot, SamplesPerCell=40, LineWidth=1.5)
title("Solved parameter-dependent Lyapunov matrix")
```

The figure contains one curve for each entry of the solved 2-by-2 matrix $P(\rho)$, sampled on the parameter interval. It is a diagnostic visualization of the feasible solution, not a certificate beyond the coefficient-wise constraints supplied to the solver. The constant-case result is retained to exercise the comparison in the smoke test; only a MOSEK solve is used there to certify its intended infeasibility distinction.

8.3 Rate-Dependent Block LMI Solver Smoke

This example is adapted from `+tests/+pdlmi/test_solver_smoke.m`. The goal is to find a parameter-dependent $P(\rho)$ and scalar γ such that, on every Bernstein coefficient and rate vertex,

$$\begin{bmatrix} \dot{P} + PA + A^\top P & PB & C^\top \\ B^\top P & -\gamma I & D^\top \\ C & D & -\gamma I \end{bmatrix} \preceq 0, \qquad P(\rho) \succeq 0.$$

The derivative term $\dot{P}$ is built with `rhodiff(P, [-1 1])`. The comparison operators create `pdlmi` wrappers, and `toYalmip` hands the coefficient-wise constraints to YALMIP. In the second listing, the `fprintf` call prints the solver- and tolerance-dependent numeric value of γ. For this model, `numel(E1.Constraints)` returns (6), and `numel(E2.Constraints)` returns (2).

MATLAB input

```
yalmip('clear')
grid = linspace(0, 1, 2);
A = pdmat(grid, @(x) [−1, 0.5; −1, −2] + ...
    x * [−1.3, −20; 2, −10], Degree=1);
B = pdmat(grid, @(x) [1, −4; −1, −1] + ...
    x * [2.2, 0.5; −6, −5], Degree=1);
Cmat = eye(2);
Dmat = zeros(2);

P = pdvar(2, grid);
diffP = rhodiff(P, [−1 1]);
gamma = pdvar(1, grid, Degree=0);
E1 = [diffP + P * A + A' * P, P * B, Cmat';
      B' * P, −gamma * eye(2), Dmat';
      Cmat, Dmat, −gamma * eye(2)] <= 0;
E2 = P >= 0;
numel(E1.Constraints)
numel(E2.Constraints)
```

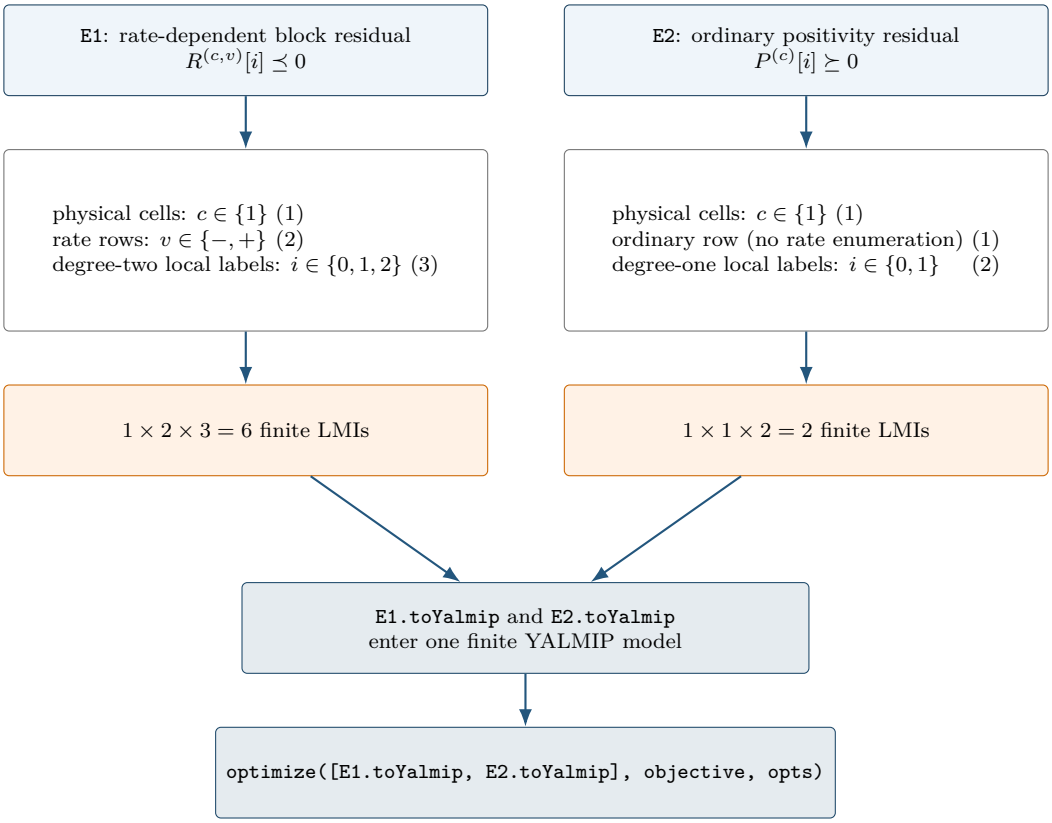
Output

```
ans =
     6

ans =
     2
```

MATLAB input (continued)

```
objective = gamma.LocalValues{1}{1};
solver = 'lmilab';
if exist('mosekopt', 'file') ~= 0
    solver = 'mosek';
end
opts = sdpsettings('solver', solver, 'verbose', 0);
sol = optimize([E1.toYalmip, E2.toYalmip], objective, opts);
assert(sol.problem == 0, sol.info)
gammaValue = value(objective);
assert(isfinite(gammaValue))
fprintf("Optimal H—infinity gamma: %.6g\n", gammaValue)
```

The constraint counts above follow directly from the three independent assembly axes: physical-cell identity, rate-row identity, and local Bernstein label. In this tested example the two wrappers use the same physical grid but different rate-row and degree structures.



The numbers (6) and (2) are certificate-assembly counts: they state how many finite semidefinite constraints represent `E1` and `E2`. They are not evidence that the optimization succeeds. Feasibility is checked separately by `sol.problem`, and the finite objective is checked after the shared `optimize` call.

A

Bernstein Basis Mathematics

This appendix develops the representation used by the package from the one-dimensional degree-one case to cell-local tensor polynomials. The reader needs only scalar polynomials, matrices, and elementary calculus. Every conversion, elevation, product, derivative, and subdivision below is exact; none is a sampling approximation. Farouki gives a broader historical and numerical account [1].

A.1 Convex Combinations Before Polynomials

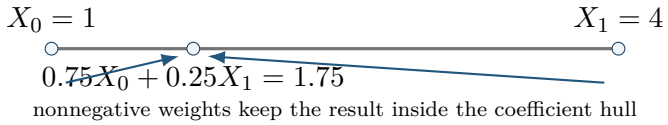
A convex combination of numbers or matrices has the form

$$X = \sum_{i=0}^{m} w_i X_i, \qquad w_i \geq 0, \qquad \sum_{i=0}^{m} w_i = 1.$$

For scalars, X lies between the smallest and largest X_i. For symmetric matrices, if every $X_i \succeq 0$, then $X \succeq 0$, because

$$z^{\mathsf{T}} X z = \sum_i w_i \, z^{\mathsf{T}} X_i z \geq 0 \quad \text{for every } z.$$

Bernstein bases are useful because their basis values are precisely such weights at every point of a cell.



The implication is one-way. A polynomial may be positive everywhere while one Bernstein coefficient is negative; Appendix B uses that fact to motivate stronger certificates.

A.2 Degree-One Interpolation On One Physical Cell

Let a physical cell be $[\rho_{c-1}, \rho_c]$, with width $h_c = \rho_c - \rho_{c-1} > 0$. Its forward local coordinate is

$$\alpha = \frac{\rho - \rho_{c-1}}{h_c} \in [0, 1].$$

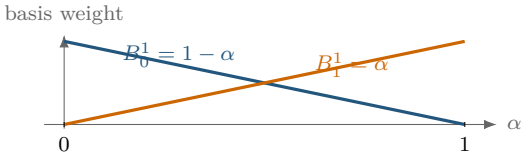
The degree-one Bernstein basis is

$$B_0^1(\alpha) = 1 - \alpha, \qquad B_1^1(\alpha) = \alpha.$$

Therefore

$$M_c(\rho) = (1 - \alpha) C^{(c)}[0] + \alpha C^{(c)}[1].$$

At the lower physical face, $\alpha = 0$ and the value is $C^{(c)}[0]$; at the upper face, $\alpha = 1$ and the value is $C^{(c)}[1]$. Between them the matrix follows the straight segment joining those two coefficient matrices.



For example, $C^{(c)}[0] = 2$ and $C^{(c)}[1] = 4$ give $M_c(0.25) = 0.75(2) + 0.25(4) = 2.5$, exactly as in section 4.3.

A.3 Higher Degree And The Partition Of Unity

For degree m, define

$$B_i^m(\alpha) = \binom{m}{i}(1-\alpha)^{m-i}\alpha^i, \qquad i = 0, \ldots, m.$$

Each basis function is nonnegative on $[0, 1]$. The binomial theorem gives

$$\sum_{i=0}^{m} B_i^m(\alpha) = ((1-\alpha) + \alpha)^m = 1.$$

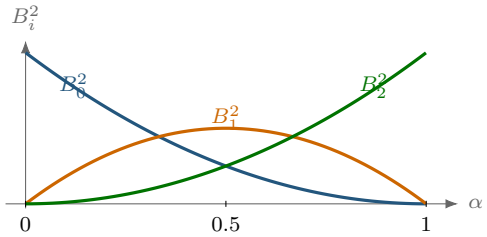
Thus

$$M_c(\alpha) = \sum_{i=0}^{m} B_i^m(\alpha) C^{(c)}[i]$$

is a convex combination of its local coefficient matrices.

For degree two,

$$B_0^2 = (1-\alpha)^2, \qquad B_1^2 = 2(1-\alpha)\alpha, \qquad B_2^2 = \alpha^2.$$



At every α, the three plotted heights are nonnegative and sum to one.

The label i identifies a basis position inside the selected physical cell. It is not a physical grid-node subscript.

A.4 Worked Conversion: $1 + 2\alpha + 3\alpha^2$

Consider

$$p(\alpha) = 1 + 2\alpha + 3\alpha^2.$$

Seek degree-two Bernstein coefficients b_0, b_1, b_2:

$$p = b_0(1-\alpha)^2 + 2b_1(1-\alpha)\alpha + b_2\alpha^2.$$

Matching powers of α gives

$$b_0 = 1, \qquad b_1 = 2, \qquad b_2 = 6.$$

Indeed,

$$1B_0^2 + 2B_1^2 + 6B_2^2 = (1 - 2\alpha + \alpha^2) + (4\alpha - 4\alpha^2) + 6\alpha^2 = 1 + 2\alpha + 3\alpha^2.$$

The three values $1, 2, 6$ are coefficients of basis functions, not samples at three equally spaced physical points. Only the two endpoint coefficients are endpoint values. At $\alpha = 1/2$,

$$p(1/2) = \tfrac{1}{4}(1) + \tfrac{1}{2}(2) + \tfrac{1}{4}(6) = 2.75.$$

MATLAB input

```
p = pdmat({[0 1]}, {1, 2, 6}, Degree=2);
fprintf("p(0.5)=%.2f\n", p.evaluate(0.5));
disp(bernsteinTable(p, "oneLine"))
```

Output

```
p(0.5)=2.75
    CellSubscript                    Expression
    -------------    ------------------------------------------

        {[1]}        "(1-alpha)^2*1 + 2(1-alpha)alpha*2 + alpha^2*6"
```

A.5 Exact Power–Bernstein Conversion

More generally, use brackets for algebraic coefficient labels as well:

$$p(\alpha) = \sum_{j=0}^{n} a[j]\alpha^j = \sum_{k=0}^{n} b[k]B_k^n(\alpha).$$

Power coefficients convert to Bernstein coefficients by

$$b[k] = \sum_{j=0}^{k} \frac{\binom{k}{j}}{\binom{n}{j}} a[j], \qquad k = 0, \ldots, n.$$

The inverse map is

$$a[j] = \binom{n}{j} \sum_{i=0}^{j} (-1)^{j-i} \binom{j}{i} b[i], \qquad j = 0, \ldots, n.$$

These are changes of basis inside the same degree-n polynomial space. For matrix polynomials they act entrywise. For tensor polynomials they act along one coordinate axis at a time.

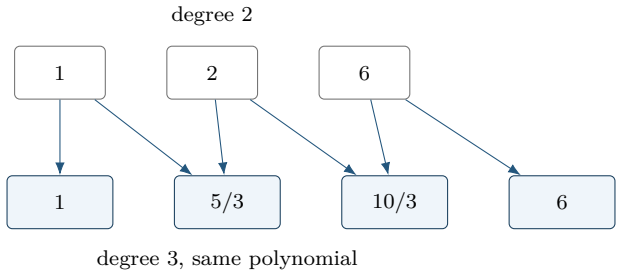
A.6 Lossless Degree Elevation

A polynomial on physical cell (c) with degree (m) also has a Bernstein representation of every higher degree. One elevation step gives

$$\widehat{C}^{(c)}[0] = C^{(c)}[0], \qquad \widehat{C}^{(c)}[k] = \frac{k}{m+1} C^{(c)}[k-1] + \left(1 - \frac{k}{m+1}\right) C^{(c)}[k], \qquad \widehat{C}^{(c)}[m+1] = C^{(c)}[m].$$

For the worked coefficients $[1, 2, 6]$, elevation from degree two to degree three produces

$$\left[1, \frac{5}{3}, \frac{10}{3}, 6\right].$$



Direct elevation from m to $M \geq m$ is

$$\widehat{C}^{(c)}[k] = \sum_{i=\max(0,k-M+m)}^{\min(m,k)} C^{(c)}[i] \frac{\binom{m}{i}\binom{M-m}{k-i}}{\binom{M}{k}}.$$

The new coefficients are fixed convex combinations of the old ones. Elevation therefore changes representation but adds no decision freedom. Constructing a new higher-degree `pdvar` does enlarge the decision parameterization.

A.7 Physical Tensor Grids And Local Tensor Labels

Suppose direction s has physical nodes

$$\rho_{s,0} < \rho_{s,1} < \cdots < \rho_{s,N_s-1}.$$

The counts N_s may differ. A physical node is identified by the ℓ-vector $\boldsymbol{k}$; a physical cell is identified separately by $\boldsymbol{c} = (c_1, \ldots, c_\ell)$. On that cell,

$$\alpha_s = \frac{\rho_s - \rho_{s,c_s-1}}{\rho_{s,c_s} - \rho_{s,c_s-1}}, \qquad s = 1, \ldots, \ell.$$

For the package's scalar degree m,

$$M_{\boldsymbol{c}}(\boldsymbol{\alpha}) = \sum_{\boldsymbol{i} \in \{0,\ldots,m\}^\ell} B^m_{\boldsymbol{i}}(\boldsymbol{\alpha}) C^{(\boldsymbol{c})}[\boldsymbol{i}], \qquad B^m_{\boldsymbol{i}} = \prod_{s=1}^{\ell} B^m_{i_s}(\alpha_s).$$

There are $\prod_s (N_s - 1)$ physical cells and $(m+1)^\ell$ local coefficients per cell.

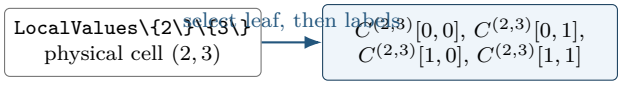
MATLAB input

```
obj = pdbase({[0 1 2], [10 20 30 40]}, [1 1], 1);
fprintf("physical cells=%d, local coefficients/cell=%d\n", ...
    obj.ncell(), obj.ncoeff());
disp(obj.lbls())
```

Output

```
physical cells=6, local coefficients/cell=4
     0     0
     0     1
     1     0
     1     1
```

The nested tree `LocalValues\{c1\}\{c2\}...\{cell\}` follows physical-cell coordinates. Only after reaching a leaf does the flat coefficient cell follow the `lbls` order. Earlier dimensions vary more slowly in both `cells` and `lbls`.



A.8 Products As Ordered Weighted Convolution

Fix a physical cell (c), and let

$$P = \sum_{i=0}^{m} B_i^m P^{(c)}[i], \qquad Q = \sum_{j=0}^{n} B_j^n Q^{(c)}[j].$$

The basis-product identity

$$B_i^m B_j^n = \frac{\binom{m}{i}\binom{n}{j}}{\binom{m+n}{i+j}} B_{i+j}^{m+n}$$

gives

$$(PQ)^{(c)}[k] = \sum_{i+j=k} P^{(c)}[i] Q^{(c)}[j] \frac{\binom{m}{i}\binom{n}{j}}{\binom{m+n}{k}}.$$

This is an ordered matrix convolution. In general $P^{(c)}[i]Q^{(c)}[j] \neq Q^{(c)}[j]P^{(c)}[i]$.

As a scalar example, degree-one coefficients $P = [1, 3]$ and $Q = [2, 4]$ produce

$$PQ = [\,1 \cdot 2,\ (1 \cdot 4 + 3 \cdot 2)/2,\ 3 \cdot 4\,] = [2, 5, 12].$$

MATLAB input

```
P = pdmat({[0 1]}, {1, 3}, Degree=1);
Q = pdmat({[0 1]}, {2, 4}, Degree=1);
R = P * Q;
c = R.coeffs(1);
fprintf("product degree=%d\n", R.Degree);
disp([c{:}])
```

Output

```
product degree=2
     2     5    12
```

For tensor labels,

$$(PQ)^{(c)}[\boldsymbol{k}] = \sum_{\boldsymbol{i}+\boldsymbol{j}=\boldsymbol{k}} P^{(c)}[\boldsymbol{i}] Q^{(c)}[\boldsymbol{j}] \prod_{s=1}^{\ell} \frac{\binom{m}{i_s}\binom{n}{j_s}}{\binom{m+n}{k_s}}.$$

Labels add componentwise; physical-cell identity does not change.

A.9 Differentiation And Rate Vertices

On physical cell (c) of width h_c,

$$\frac{\partial P}{\partial \rho} = \frac{m}{h_c} \sum_{i=0}^{m-1} (C^{(c)}[i+1] - C^{(c)}[i]) B_i^{m-1}(\alpha).$$

For the worked polynomial $[1, 2, 6]$ on $[0, 2]$, $m = 2$ and $h = 2$, so the derivative coefficients are

$$\left[\frac{2}{2}(2-1), \frac{2}{2}(6-2)\right] = [1, 4].$$

This represents $dp/d\rho = 1 + 3\alpha$.

In ℓ dimensions, differentiation with respect to ρ_s replaces each coefficient by

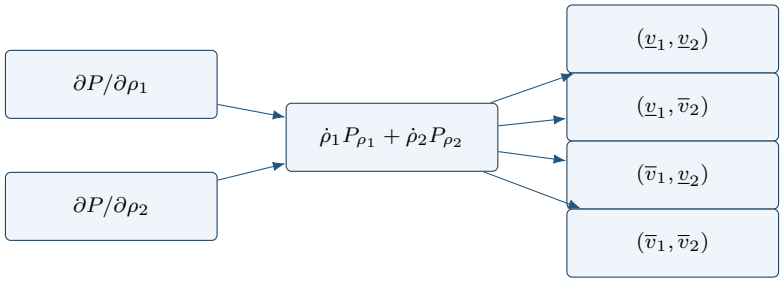
$$\frac{m}{h_s}\left(C^{(\boldsymbol{c})}[\boldsymbol{i} + \boldsymbol{e}_s] - C^{(\boldsymbol{c})}[\boldsymbol{i}]\right),$$

reduces the degree along direction s, and leaves the other label components unchanged. The implementation elevates partial derivatives to a common tensor degree before adding them.

Finally,

$$\dot{P} = \sum_{s=1}^{\ell} \frac{\partial P}{\partial \rho_s} \dot{\rho}_s.$$

This matrix expression is affine in $\dot{\boldsymbol{\rho}}$. Every value inside the rate box is therefore a convex combination of the 2^ℓ vertex matrices. Because the positive- and negative-semidefinite cones are convex, the requested semidefinite sign holds on the whole rate box whenever it holds at every vertex. `rhodiff` stores one row for each vertex.



A.10 de Casteljau Evaluation And Exact Subdivision

de Casteljau recursion evaluates a Bernstein polynomial using only affine combinations. On a fixed physical cell (c), begin with $C^{(c,0)}[i] = C^{(c)}[i]$ and define

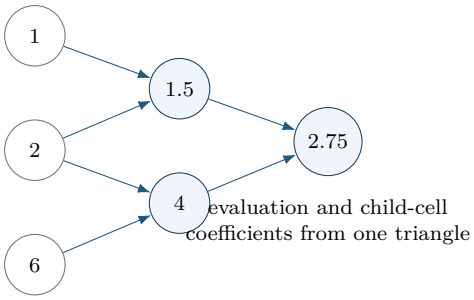
$$C^{(c,r)}[i] = (1-\tau)C^{(c,r-1)}[i] + \tau C^{(c,r-1)}[i+1], \qquad r = 1, \ldots, m.$$

The value at $\alpha = \tau$ is $C^{(c,m)}[0]$.

For $[1, 2, 6]$ and $\tau = 1/2$, the recursion is

$$[1, 2, 6] \longrightarrow [1.5, 4] \longrightarrow [2.75].$$

The left edge of this triangle gives exact coefficients $[1, 1.5, 2.75]$ on the left subinterval; the right edge gives $[2.75, 4, 6]$ on the right.



Tensor evaluation or subdivision applies the same recursion along one parameter direction at a time. Subdivision changes the physical-cell partition and can tighten coefficient hulls while preserving the polynomial. The package uses exact coefficient transformations internally where documented, but it does not currently expose a public de Casteljau, subdivision, or adaptive-refinement API.

A.11 What Changes, And What Does Not

The following distinctions prevent several common modeling mistakes.

Operation	Changes	Preserves
Basis conversion	coefficient coordinates	polynomial, degree, physical cell
Degree elevation	Bernstein degree and coefficient list	polynomial, physical grid, decision freedom
Grid subdivision	physical-cell partition and local coefficients	polynomial on the original domain
Higher-degree `pdvar`	number of decision coefficients	grid and matrix size
Multiplication	polynomial degree and coefficients	physical-cell identity
Differentiation	degree and coefficient values	physical-cell partition before rate rows

These representation facts support the finite certificates in the next appendix. They do not by themselves select a solver or prove that one certificate is necessary.

B

Polynomial Matrix Sign Certificates On A Hypercube

This appendix starts with positive semidefinite matrices, builds SOS and Gram representations, and then explains the three certificate choices implemented by `pdlmi`. Background constructions are included so a new reader can recognize the mathematical landscape; only explicitly marked methods are package APIs.

For a residual $F \preceq 0$, define $S = -F$. For $F \succeq 0$, define $S = F$. The common problem is

$$S(\boldsymbol{\alpha}) \succeq 0 \quad \text{for every } \boldsymbol{\alpha} \in [0,1]^{\ell}.$$

All Gram constraints below prove non-strict inequalities. A strict model must state its own margin, for example $S - \varepsilon I \succeq 0$. The package adds no hidden margin.

B.1 Positive Semidefinite Matrices And LMIs

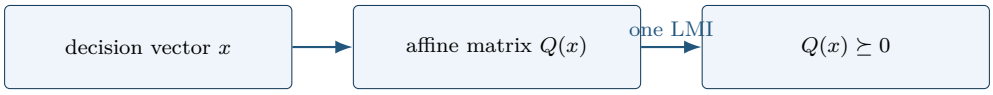
A real symmetric matrix Q is positive semidefinite when

$$z^{\mathsf{T}} Q z \geq 0 \qquad \text{for every vector } z.$$

An LMI is an affine matrix expression constrained to the PSD cone:

$$Q(x) = Q_0 + x_1 Q_1 + \cdots + x_r Q_r \succeq 0.$$

The unknowns enter affinely, so this is a semidefinite program. A polynomial matrix sign condition appears infinite because it asks for one PSD matrix at every point. A certificate replaces those pointwise conditions by finitely many PSD matrices and affine coefficient identities.



B.2 SOS And Gram Matrices

A scalar polynomial is a sum of squares (SOS) if

$$p(z) = q_1(z)^2 + \cdots + q_s(z)^2.$$

Every SOS polynomial is nonnegative. Choose a polynomial basis vector $v_d(z)$. The Gram form

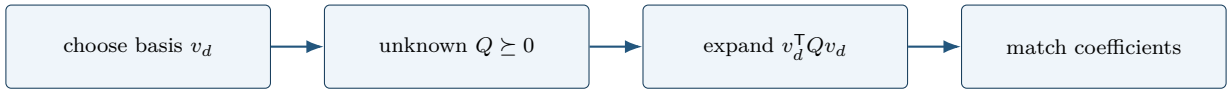
$$p(z) = v_d(z)^{\mathsf{T}} Q v_d(z), \qquad Q \succeq 0,$$

is SOS because a factorization $Q = L^{\mathsf{T}} L$ yields $p = \|L v_d\|_2^2$. Expanding the Gram form and matching every coefficient of p gives equations linear in the entries of Q. Thus an SOS search is a finite PSD constraint plus affine equalities.

For a symmetric n-by-n polynomial matrix, set

$$Z_d(z) = v_d(z) \otimes I_n, \qquad S(z) = Z_d(z)^{\mathsf{T}} Q Z_d(z), \quad Q \succeq 0.$$

Then $y^{\mathsf{T}} S(z) y = (Z_d y)^{\mathsf{T}} Q (Z_d y) \geq 0$ for every y, so $S(z) \succeq 0$. Matrix-SOS relaxations are developed in [9].



B.3 Full-Space SOS

An unweighted Gram form proves $S(z) \succeq 0$ on all of $\mathbb{R}^\ell$. That can be much stronger than positivity only on a box. For example, $\alpha(1-\alpha)$ is nonnegative on $[0,1]$ but negative outside it. The package does not expose a general full-space SOS parser; this construction is background for understanding weighted certificates.

B.4 Direct Bernstein Coefficients

On physical cell $\boldsymbol{c}$ and one rate row, write

$$S(\boldsymbol{\alpha}) = \sum_{\boldsymbol{i}} B^m_{\boldsymbol{i}}(\boldsymbol{\alpha}) C^{(\boldsymbol{c})}[\boldsymbol{i}].$$

Because the basis weights are nonnegative and sum to one,

$$C^{(\boldsymbol{c})}[\boldsymbol{i}] \succeq 0 \ \forall \boldsymbol{i} \quad \Longrightarrow \quad S(\boldsymbol{\alpha}) \succeq 0.$$

This is the package default. It introduces no Gram variables and creates one constraint per physical cell, rate row, and local label. The converse is false.

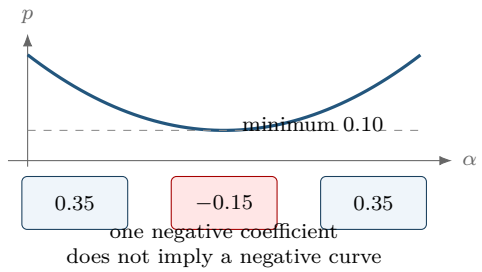
Consider

$$p(\alpha) = \alpha^2 - \alpha + 0.35 = (\alpha - 0.5)^2 + 0.10.$$

It is strictly positive, with minimum 0.10, but its degree-two Bernstein coefficients are

$$[\,0.35,\ -0.15,\ 0.35\,].$$

The negative middle coefficient defeats the direct certificate without making the polynomial negative.



The same polynomial has the PSD Bernstein Gram representation

$$\zeta_1 = \begin{bmatrix} 1-\alpha \\ \alpha \end{bmatrix}, \qquad Q = \begin{bmatrix} 0.35 & -0.15 \\ -0.15 & 0.35 \end{bmatrix} \succeq 0, \qquad p = \zeta_1^{\mathsf{T}} Q \zeta_1.$$

The eigenvalues of Q are 0.20 and 0.50. A complete Gram certificate can therefore succeed when coefficient-wise signs fail.

B.5 Pólya As Lossless Elevation

Classical Pólya theory concerns homogeneous polynomials strictly positive on a simplex [12]. For one box coordinate,

$$\lambda_0 = 1-\alpha, \quad \lambda_1 = \alpha, \quad \lambda_0 + \lambda_1 = 1.$$

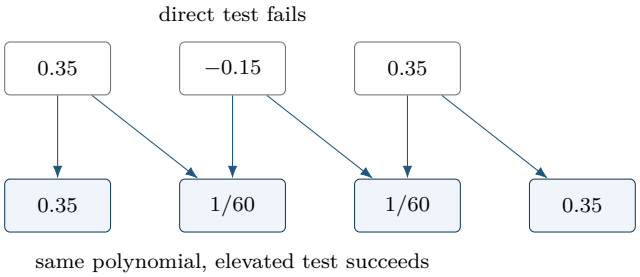
For ℓ coordinates, the implemented multiplier is

$$\prod_{s=1}^{\ell} (\lambda_{s,0} + \lambda_{s,1})^d = 1.$$

Regrouping is exactly Bernstein degree elevation: the represented residual does not change.
For the worked coefficients,

$$[\,0.35, -0.15, 0.35\,] \longrightarrow \left[0.35, \frac{1}{60}, \frac{1}{60}, 0.35\right]$$

after one elevation. Every new coefficient is positive, so the elevated coefficient test succeeds.



`applyPolya(d)` applies one scalar increment in every parameter direction and then coefficient constraints to every cell and rate row. It adds neither Gram variables nor decision freedom. A fixed-level failure is inconclusive; the package makes no tensor-box completeness claim.

B.6 Exact Interval Markov–Lukács Forms

In one variable, interval nonnegativity has exact parity-specific descriptions [6, 7]. If $\deg p \le 2r$,

$$p \ge 0 \text{ on } [0,1] \iff p = s_0 + \alpha(1-\alpha)s_1,$$

where s_0, s_1 are SOS of degrees at most $2r$ and $2r-2$. If $\deg p \le 2r+1$,

$$p \ge 0 \text{ on } [0,1] \iff p = (1-\alpha)s_L + \alpha s_U,$$

where s_L, s_U are SOS of degree at most $2r$.

For polynomial matrices, with

$$\mathcal{Z}_d(\alpha) = \begin{bmatrix} B_0^d I_n & \cdots & B_d^d I_n \end{bmatrix}^{\mathsf{T}},$$

the even and odd forms are

$$S = \mathcal{Z}_r^{\mathsf{T}} Q_0 \mathcal{Z}_r + \alpha(1-\alpha)\mathcal{Z}_{r-1}^{\mathsf{T}} Q_1 \mathcal{Z}_{r-1}, \qquad Q_0, Q_1 \succeq 0,$$

and

$$S = (1-\alpha)\mathcal{Z}_r^{\mathsf{T}} Q_L \mathcal{Z}_r + \alpha \mathcal{Z}_r^{\mathsf{T}} Q_U \mathcal{Z}_r, \qquad Q_L, Q_U \succeq 0.$$

The second even block is absent at $r = 0$. See [8, 9] for polynomial-matrix context.

even $2r$	odd $2r+1$
unweighted Gram + $\alpha(1-\alpha)$-weighted Gram	$(1-\alpha)$ lower-face Gram + α upper-face Gram

Partition Q into n-by-n blocks Q_{ij}. The Bernstein coefficient at label k of $\alpha^a(1-\alpha)^b \mathcal{Z}_d^{\mathsf{T}} Q \mathcal{Z}_d$, expressed at total degree $2d + a + b$, is

$$\mathcal{G}_k^{d,a,b}(Q) = \frac{1}{\binom{2d+a+b}{k}} \sum_{\substack{0 \le i,j \le d \\ i+j=k-a}} \binom{d}{i}\binom{d}{j} Q_{ij}.$$

Equating this affine map to each target coefficient gives exact linear identities; $Q \succeq 0$ supplies the LMIs. For one parameter, `applyFullBoxPreorder` implements these exact parity forms. Its minimum absolute order is $\lfloor m/2 \rfloor$.

B.7 Putinar Modules And Schmüdgen Preorderings

For

$$K = \{z : g_1(z) \ge 0, \ldots, g_q(z) \ge 0\},$$

a Putinar quadratic module has the form

$$\sigma_0 + \sum_{j=1}^{q} \sigma_j g_j, \qquad \sigma_j \text{ SOS}.$$

Under the Archimedean hypothesis, every polynomial strictly positive on K has such a representation at some degree [10]. Compactness alone is not that theorem statement. The package does not implement a general Putinar interface.

A Schmüdgen full preordering includes every square-free generator product:

$$p = \sum_{\boldsymbol{\nu} \in \{0,1\}^q} \sigma_{\boldsymbol{\nu}} \prod_{j=1}^{q} g_j^{\nu_j}, \qquad \sigma_{\boldsymbol{\nu}} \text{ SOS}.$$

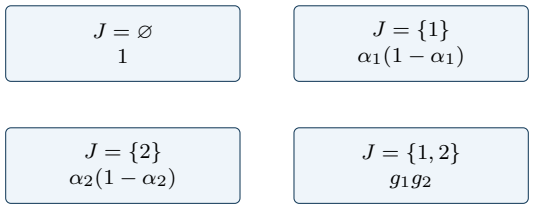
Strict positivity on a compact basic closed semialgebraic set has such a representation [11]. A finite truncation is still a sufficient certificate, and up to 2^q multiplier blocks may be required.

B.8 The Implemented Multivariate Full Box Preordering

For the hypercube, use

$$g_s(\boldsymbol{\alpha}) = \alpha_s(1-\alpha_s), \qquad s = 1, \dots, \ell.$$

For two parameters, the four subset products are $1, g_1, g_2, g_1 g_2$.



For $\ell \geq 2$ and absolute order r, define

$$\mathcal{Z}_{\boldsymbol{d}} = \left(\bigotimes_{s=1}^{\ell} \begin{bmatrix} B_0^{d_s} & \cdots & B_{d_s}^{d_s} \end{bmatrix}^{\mathsf{T}} \right) \otimes I_n.$$

The implemented form is

$$S = \sum_{J \subseteq \{1,\dots,\ell\}} \left(\prod_{s \in J} g_s \right) \mathcal{Z}_{r\mathbf{1}-\boldsymbol{\chi}_J}^{\mathsf{T}} Q_J \mathcal{Z}_{r\mathbf{1}-\boldsymbol{\chi}_J}, \qquad Q_J \succeq 0.$$

Terms with negative tensor degree are omitted. A present block has dimension

$$n \prod_{s=1}^{\ell} \left(r + 1 - \mathbf{1}_{s \in J} \right).$$

Every physical cell and active rate row gets independent dense Gram blocks and exact coefficient identities. The minimum order is $\lceil m/2 \rceil$ for $\ell \geq 2$. This is a specific dense Bernstein realization of the complete box preordering, not a full-space SOS, Putinar-only, sparse, or general-domain parser. A fixed multivariate order is sufficient rather than exact.

MATLAB input

```
yalmip('clear')
P = pdvar(1, {[0 1]}, Degree=2);
direct = P >= 0;
polya = direct.applyPolya(1);
box = direct.applyFullBoxPreorder();
fprintf("direct: constraints=%d\n", numel(direct.Constraints));
fprintf("Polya: increment=%d, constraints=%d\n", ...
    polya.PolyaDegree, numel(polya.Constraints));
fprintf("full box: order=%d, constraints=%d\n", ...
    box.FullBoxOrder, numel(box.Constraints));
```

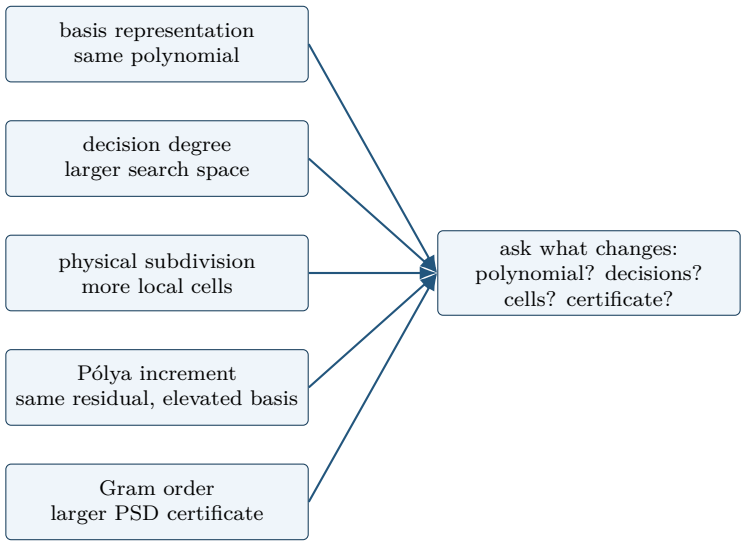
Output

```
direct: constraints=3
Polya: increment=1, constraints=4
full box: order=1, constraints=5
```

Selections rebuild from the stored original `Residual` and `Relation`. Reapplying Pólya or full-box replaces the previous selection; the certificates never compound.

B.9 Five Different Modeling Choices

These choices act on different parts of the problem.



Choice	Changes	Does not mean
basis conversion	coefficient coordinates	more decision freedom
decision degree	independent decision coefficients	certificate order
subdivision	physical cells and local hulls	higher polynomial degree
Pólya degree	elevated residual coefficients	Gram variables
full-box order	Gram bases and PSD block sizes	decision degree

When a certificate fails, do not infer continuous infeasibility. Decide separately whether the model needs a richer decision degree, a finer physical grid, a larger Pólya increment, or a higher full-box order.

B.10 Implemented Boundary

Implemented public behavior	Background only
direct coefficient assembly	general full-space SOS parser
uniform `applyPolya([d])`	direction-wise or automatic Pólya search
dense `applyFullBoxPreorder([r])`	general Putinar interface
exact one-dimensional parity forms	general-domain Schmüdgen parser
all cells and active rate rows	sparse Gram selection or adaptive refinement
explicit user margins	package-owned strictness or solver policy

Only the left column is callable package behavior. The background theorems explain alternative cones; they do not expand the MATLAB API.

References

[1] R. T. Farouki, "The Bernstein polynomial basis: A centennial retrospective," *Computer Aided Geometric Design*, vol. 29, no. 6, pp. 379–419, 2012. doi:10.1016/j.cagd.2012.03.001.

[2] M. Zettler and J. Garloff, "Robustness analysis of polynomials with polynomial parameter dependency using Bernstein expansion," *IEEE Transactions on Automatic Control*, vol. 43, no. 3, pp. 425–431, 1998. doi:10.1109/9.661615.

[3] I. Masubuchi, A. Kume, and E. Shimemura, "Spline-type solution to parameter-dependent LMIs," in *Proceedings of the 37th IEEE Conference on Decision and Control*, vol. 2, 1998. doi:10.1109/CDC.1998.758549.

[4] Y. Xu and F. Jabbari, "Spline-based parameter varying output feedback synthesis with improved L_2 gain," in *2024 IEEE 63rd Conference on Decision and Control*, pp. 1455–1460, 2024. doi:10.1109/CDC56724.2024.10886461.

[5] G. Chesi, "LMI techniques for optimization over polynomials in control: a survey," *IEEE Transactions on Automatic Control*, vol. 55, no. 11, pp. 2500–2510, 2010. doi:10.1109/TAC.2010.2046926.

[6] F. Lukács, "Verschärfung des ersten Mittelwertsatzes der Integralrechnung für rationale Polynome," *Mathematische Zeitschrift*, vol. 2, pp. 295–305, 1918. EuDML record.

[7] E. de Klerk, R. Hess, and M. Laurent, Improved convergence rates for Lasserre-type hierarchies of upper bounds for box-constrained polynomial optimization, *SIAM Journal on Optimization*, vol. 27, no. 1, pp. 347–367, 2017. doi:10.1137/16M1065264.

[8] E. M. Aylward, S. M. Itani, and P. A. Parrilo, Explicit SOS decompositions of univariate polynomial matrices and the Kalman–Yakubovich–Popov lemma, in *Proceedings of the 46th IEEE Conference on Decision and Control*, pp. 5660–5665, 2007. author manuscript.

[9] C. W. Scherer and C. W. J. Hol, Matrix sum-of-squares relaxations for robust semi-definite programs, *Mathematical Programming*, vol. 107, pp. 189–211, 2006. doi:10.1007/s10107-005-0684-2.

[10] M. Putinar, "Positive polynomials on compact semi-algebraic sets," *Indiana University Mathematics Journal*, vol. 42, no. 3, pp. 969–984, 1993. JSTOR 24897130.

[11] K. Schmüdgen, "The K-moment problem for compact semi-algebraic sets," *Mathematische Annalen*, vol. 289, pp. 203–206, 1991. doi:10.1007/BF01446568.

[12] V. Powers and B. Reznick, "A new bound for Pólya's theorem with applications to polynomials positive on polyhedra," *Journal of Pure and Applied Algebra*, vol. 164, nos. 1–2, pp. 221–229, 2001. doi:10.1016/S0022-4049(00)00155-9.